GenAI Course Notes – June 2026

Sessions 1–8 · in date order · each session starts with its date page

DATE	SESSION & TOPIC		PAGE
6 June 2026	**Session 1**	RAG Fundamentals	2
7 June 2026	**Session 2**	Chunking & Embeddings	13
13 June 2026	**Session 3**	Building an End-to-End RAG Chain	27
14 June 2026	**Session 4**	Embedding Theory & Retrieval Methods	41
20 June 2026	**Session 5**	RAG App, Latency, Multimodal RAG & Agents Intro	53
21 June 2026	**Session 6**	Multimodal RAG Build & Redis	66
27 June 2026	**Session 7**	Agent Frameworks: LangGraph, CrewAI, AutoGen	79
28 June 2026	**Session 8**	Project: Video Compliance Auditor on Azure	91

SESSION 1

Saturday, 6 June 2026

RAG Fundamentals

RAG Notes – Session 1 (6 June 2026)

Oct 14, 2026 · @Vljay

1. What RAG is and why we need it

RAG = Retrieval-Augmented Generation. It lets an LLM answer from *your* data (company policies, internal docs, databases) by first retrieving relevant passages and then generating an answer grounded in them. The lecture estimates that most real-world GenAI work (around 80%) is some form of RAG.

Simple example: an employee asks "What is our leave policy?" A plain LLM has never seen your HR handbook, so it guesses. A RAG system searches the company documents, finds the leave-policy section, and answers from it.

The three core LLM limitations RAG solves (interview answer)

1. **Knowledge cutoff** – an LLM only knows what was in its training data up to a certain date. It cannot know a policy HR published last week.
2. **Hallucination** – an LLM can confidently produce wrong answers. Ambiguity makes this worse (e.g. "bank" = financial institution vs. river bank) when there is no grounding context.
3. **No access to private data** – the LLM cannot see your SharePoint, emails, PDFs, SQL/MongoDB databases, Slack, etc.

Full list of benefits (for notes)

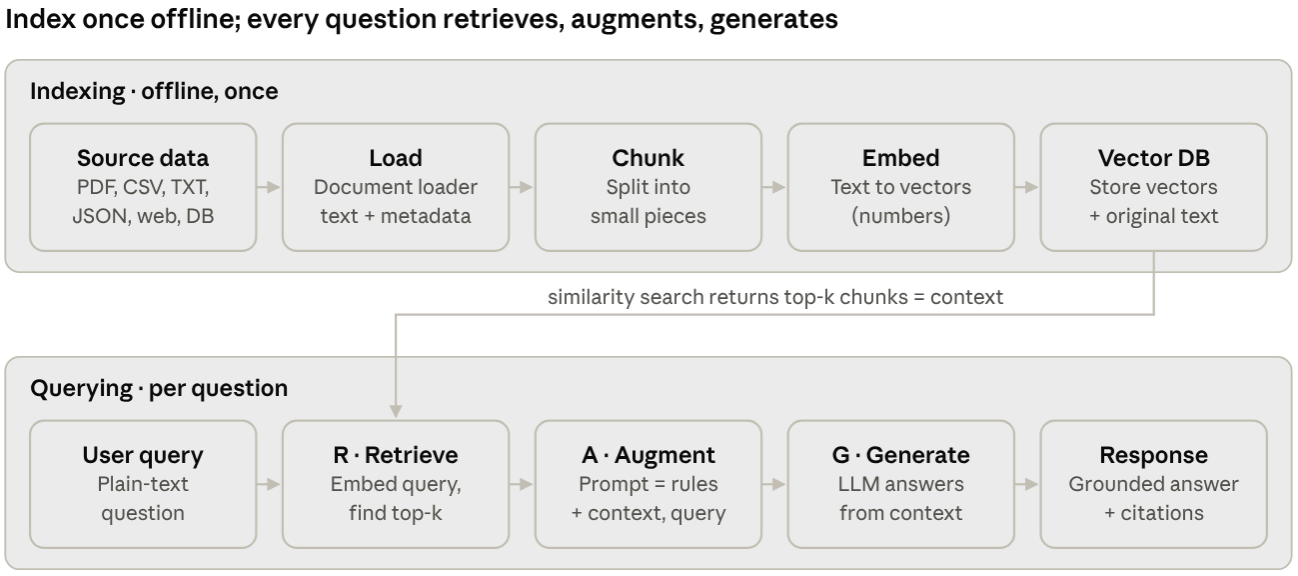
#	Benefit	What it means
1	Beats knowledge cutoff	Retrieves recent/real-time info the model was never trained on
2	Privacy and compliance	Sensitive data stays in a controlled retrieval store instead of being baked into model weights
3	Less hallucination	Answers are grounded in retrieved text; can show **source attribution / citations**
4	Domain-specific accuracy	Access to specialised knowledge (e.g. an income-tax bot, an HR-only bot) that general LLMs lack
5	Long-tail entities	LLMs do poorly on rare facts (an obscure professor vs. Einstein); retrieval fills the gap

#	Benefit	What it means
6	Plugs knowledge gaps	Retrieves external knowledge, conversation history and in-context examples to help with new tasks
7	Cheaper than fine-tuning	Try RAG first; fine-tune only if RAG does not meet quality expectations

One-line definition to remember: RAG = search your own data (retrieval) + enrich the prompt with what you found (augmentation) + let the LLM write the answer (generation).

2. RAG architecture end to end

RAG runs in two phases: **indexing** prepares your documents once, and **querying** runs for every user question.



RAG architecture · indexing + querying

The vector DB is the bridge: everything built offline is searched online.

Phase 1 – Indexing (document processing, offline)

1. **Source data** – company-owned data the LLM cannot see: PDF, Word, Excel/CSV, TXT, JSON/YAML, Outlook, SQL/MongoDB, SharePoint, web pages, social data. Provided by the client or stakeholders.
2. **Load** – a LangChain document loader reads each source into `Document` objects (text + metadata).
3. **Chunk / split (transform)** – break long documents into smaller pieces so each fits the model and retrieval stays precise (like batch size in deep learning).

4. **Embed** – convert each chunk's text into a vector (a list of numbers that captures meaning).
5. **Store** – save vectors (with their original text and metadata) in a vector database such as Chroma, Pinecone or FAISS, so embedding is paid for only once.

Phase 2 – Querying (retrieval, augmentation, generation)

1. **User asks a question** in plain text.
2. **Retrieve (R)** – embed the query with the *same* embedding model, run similarity search (cosine similarity, HNSW index, top-k, e.g. k = 5) and return the most relevant chunks. These chunks are the **context**.
3. **Augment (A)** – build a prompt: instructions + context + query ("answer only from the context, otherwise say I don't know").
4. **Generate (G)** – the LLM reads the prompt and writes a human-like, grounded answer, optionally with citations.
5. **Response** goes back to the user.

Remember: in a typical project, most of the effort (the lecture says 60–70%) goes into indexing, especially loading and chunking. Poor chunking is a common cause of bad answers and latency.

3. R, A and G explained

Letter	Name	Role	Plain-English meaning
R	Retrieval	Embed the user query, run similarity search over the vector store, return the top-k chunks	Fetch/filter the relevant pieces from your knowledge base
A	Augmentation	Build a prompt = instructions + retrieved context + user question	"Enhance the prompt" with the facts the LLM needs
G	Generation	The LLM reads the augmented prompt and writes the answer	Turn raw chunks into a human-like answer

Key terms

- **Context** = the relevant documents/chunks the retriever returns (e.g. top k = 5). It is raw text, not yet a human-friendly answer.

- **Query** = the user's original question. In basic (vanilla) RAG the query is *not* rewritten; query rewriting/fusion comes later in the course.
- **Augmentation is written by the developer**, not the user. It is the system prompt baked into your RAG pipeline.

Example augmentation (system) prompt

```
You are an assistant for question-answering tasks.
Answer the question using ONLY the information in the context below.
If the answer is not in the context, say "I don't know" – do not make up an
answer.
Answer in clear, well-formatted, human language and cite the source document
where possible.

Context:
{context}

Question:
{question}
```

Why augmentation matters: it is the main guard against hallucination. It tells the LLM to stay inside the retrieved context and to admit when the answer is missing. It can also ask the model to return **citations** (which chunk/document the answer came from), using chunk metadata.

Who does what (common confusion in class): the augmentation layer only *assembles instructions*; it cannot think. The **LLM** does the actual reasoning and writing, reading only the context it was given.

4. RAG vs. other approaches

Approach	How it works	When to use	Drawback
Long-context / closed-domain QA	Paste the whole document into the prompt	One small, text-only document	Doesn't scale to many PDFs, tables, images; every question re-sends (and re-pays for) the whole document
RAG	Index once, retrieve only relevant chunks per question	Large or growing private knowledge bases; the industry default	Quality depends heavily on loading, chunking and retrieval

Approach	How it works	When to use	Drawback
Fine-tuning	Update model weights on your data	Only after RAG fails to meet quality (style, format, deep domain behaviour)	Expensive, slow to update, data gets baked into weights
Tool calling / MCP	LLM calls functions you define (APIs, live DBs)	Live or real-time data, actions	Not a replacement for document search; often combined with RAG

Rule of thumb from class: start with RAG; go to fine-tuning only if RAG isn't good enough. RAG saves the client money.

Training vs. RAG (another common confusion): the LLM is *already trained* on huge public data, so it knows how to read and write language, like someone who can already ride any bicycle. RAG does **not** retrain it. It just hands the model your internal data at question time.

5. Document loaders (LangChain)

A document loader reads raw source data into a list of LangChain `Document` objects. LangChain offers 100+ loaders (files, S3, websites, Airbyte, Unstructured.io, etc.). The lecture notes that 60–70% of RAG effort goes into indexing, and loading is its first step.

Every `Document` has two fields:

- `page_content` – the extracted text
- `metadata` – e.g. `source` path, page number, row number. Keep it: it powers citations and metadata filtering.

```
pip install -U langchain-community pypdf beautifulsoup4 jq
```

Text file (.txt, also .srt subtitles)

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader(r"data/text/file2.txt", encoding="utf-8")
docs = loader.load()          # list[Document]
print(docs[0].metadata)       # {'source': 'data/text/file2.txt'}
print(docs[0].page_content[:300])
```

The Windows error in class came from the path and file encoding. Use a raw string `r"..."`, forward slashes or `\\`, and pass `encoding="utf-8"` (or `autodetect_encoding=True`).

CSV file (one Document per row)

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader(file_path=r"data/csv/movies.csv")
docs = loader.load()
len(docs)                # e.g. 436 rows -> 436 Documents
docs[0].page_content     # "movieId: 1\ntitle: ...\ngenres: ..."
docs[:5]                 # first 5 rows
```

All files of one type in a folder

```
from langchain_community.document_loaders import DirectoryLoader, TextLoader

loader = DirectoryLoader(
    r"data/subtitles",
    glob="*.srt",              # "**/*.pdf", "*.csv", ...
    loader_cls=TextLoader,     # PyPDFLoader for PDFs, CSVLoader for CSVs
    show_progress=True,
)
docs = loader.load()
```

Mixed folder (PDF + CSV + JSON…): create one loader per file type (change `glob` and `loader_cls`), then combine the lists.

PDF (one Document per page)

```
from langchain_community.document_loaders import PyPDFLoader

docs = PyPDFLoader(r"data/pdf/attention_is_all_you_need.pdf").load()
len(docs)   # one per page
```

Images and tables inside PDFs need multimodal loaders (e.g. Unstructured), covered later.

Web page

```
from langchain_community.document_loaders import WebBaseLoader

docs = WebBaseLoader("https://example.com/article").load()
```

Sites that block scrapers (403) may need a browser-like `User-Agent` header. Pages behind a login are out of scope for document loaders, and you should respect each site's terms.

JSON

```
from langchain_community.document_loaders import JSONLoader

loader = JSONLoader(
    file_path=r"data/json/products.json",
    jq_schema=".products[]",      # iterate every product
    content_key="description",    # field that becomes page_content
)
docs = loader.load()
```

`jq_schema` must match your file's structure; if the schema doesn't match, loading fails.

Tip: don't memorise 100 loaders. Look up the one you need in the LangChain docs (Document loaders → integrations).

6. Q&A highlights

Question	Answer
Do we feed company data to the LLM every time?	No. Indexing (load → chunk → embed → store) happens once; each query only retrieves the relevant chunks. This avoids repeated embedding cost.
Why do we need an LLM if the data is internal?	Nobody can read 50,000 documents per question. The LLM turns retrieved chunks into a coherent answer. RAG cannot work without an LLM.
Who writes the augmentation prompt?	The developer, as part of the system design. The user never sees it.
How is tool calling different from RAG?	Tool calling = the LLM invokes functions you define (e.g. a live API). RAG = semantic search over indexed documents. They are often combined.
Can RAG write/update data, not just read?	Yes. The knowledge base supports add, update (e.g. a changed policy) and delete.

Question	Answer
Is there caching?	Yes. Frequently asked questions can be served from a cache instead of hitting the vector DB every time.
How do we keep confidential data safe?	Guardrails, evaluation metrics, observability and LLM-as-a-judge (later in the course), plus access control on the knowledge base.
Can one chatbot be restricted to a domain?	Yes. Build separate bots (HR, finance…) on separate knowledge bases; the HR bot replies "I don't know" to finance questions.
Why load into `Document` objects before chunking?	It gives every loader the same format and keeps metadata, which later powers citations and filtering.
Is metadata important?	Yes, mandatory in practice. It enables source citations and filtering; losing it hurts accuracy in production.
How do we trust the retrieved context?	Evaluate it: contextual precision, contextual recall, contextual relevancy. Evaluate the answer with faithfulness/hallucination, answer relevancy and groundedness.
Does it work on Azure/AWS/GCP/low-code tools?	Yes. The same architecture applies on every platform; the low-code tools run the same steps internally.
Is Google's AI search RAG?	The lecture said no, calling it plain LLM. (See correction 8 below.)

7. Expert corrections to the lecture

A few points in the session were imprecise. Use these versions in interviews.

1. **Name:** it is Retrieval-**Augmented** Generation, not "Retrieval Augmentation Generation".
2. **The LLM does not receive embeddings.** Embeddings are used only to *find* chunks (query vector vs. chunk vectors). The retrieved chunks are passed to the LLM as **plain text** inside the prompt. Nothing is "converted back" from vectors to text: each vector is stored alongside its original text.

3. **Who calls whom:** the application (e.g. a LangChain chain) orchestrates the flow: query → retriever → prompt → LLM. The LLM does not itself "connect to" the retriever (except in agentic RAG, where the LLM decides to call retrieval as a tool).
4. **Context limit:** "LLMs can't read more than 8,192 tokens" was true for some older models. Modern models handle 128K to 1M+ tokens. We still chunk for **retrieval precision, cost and latency**, and so the model isn't distracted by irrelevant text.
5. **Self-RAG is not the same as traditional/classic RAG.** Self-RAG is an advanced variant where the model critiques and decides whether to retrieve. Classic = vanilla = naive RAG.
6. **Spelling of tools:** BM25 (not BM32) · Milvus (not Milevus) · FAISS (Facebook AI Similarity Search) · multimodal RAG (not multi-model) · conversational RAG (not convertable) · `glob` (not globe) · HNSW = Hierarchical Navigable Small World, an approximate nearest-neighbour index.
7. **Fine-tuning vs. RAG:** fine-tuning mainly teaches *behaviour/style/format*; it is a poor way to inject frequently changing facts. Facts belong in RAG.
8. **Google AI Overviews / Perplexity** *do* use retrieval-augmented generation, with the web as the knowledge source. "Your own data" is the common enterprise use case, not part of the definition.
9. **Prompting alone can't remove hallucination,** but grounding + an "answer only from context" instruction + evaluation (faithfulness) reduces it substantially.

8. Course roadmap and quick revision

What's coming next

1. **Indexing deep dive:** chunking strategies → embeddings (the maths) → vector databases (Chroma, Pinecone, FAISS, Milvus).
2. **Retrieval deep dive:** semantic similarity (cosine), top-k, HNSW, BM25 keyword search, hybrid search, re-ranking, RAG-Fusion, plus pros and cons of pure semantic search.
3. **RAG types:** classic/vanilla RAG → Self-RAG → multimodal RAG → conversational RAG → Graph RAG → agentic RAG (multi-agent routing); also RAG-Token/RAG-Sequence, Fusion-in-Decoder, Speculative RAG.
4. **Production:** evaluation (DeepEval, RAGAS), LLM-as-a-judge, guardrails, observability, caching, CRUD updates to the knowledge base, fine-tuning (PEFT, LoRA, QLoRA).

60-second interview answer

> LLMs have three gaps: knowledge cutoff, hallucination and no access to private data. RAG fixes this in two phases. **Indexing** (offline): load documents, chunk them, embed the chunks and store the vectors in a vector DB. **Querying** (online): embed the user's question, retrieve the top-k most similar chunks (the context), augment a prompt with that context plus an instruction to answer only from it, and let the LLM generate a grounded answer with citations. It's cheaper and easier to update than fine-tuning, so it's the industry default.

Homework from class

- [ ] Redraw the RAG architecture from memory
- [ ] Practise every loader above (TXT, CSV, SRT, directory, PDF, web, JSON) on your own files
- [ ] Browse the LangChain document-loader integrations list
- [ ] Switch to Google Colab if local Windows path/encoding issues persist

SESSION 2

Sunday, 7 June 2026

Chunking & Embeddings

RAG Notes – Session 2 (7 June 2026)

Oct 14, 2026 · @VIjay

1. Recap: where we are

This session finishes the **chunking** step and introduces **embeddings**, steps 2 and 3 of indexing.

Phase	Steps	Status
Part 1 – Indexing	Document loader → **Chunking** → **Embedding** → Vector DB	Loader done (Session 1); chunking + embedding code today; embedding maths next
Part 2 – Querying	Retrieval → Augmentation → Generation	Next sessions

Document loaders in one look

Every LangChain loader does three things:

1. **Load** the source into memory.
2. **Parse** it (PDF, CSV, HTML, JSON, YouTube, Slack, Notion, Google Drive…).
3. **Return a unified format**: a list of `Document` objects, each with `page_content` (the text) and `metadata` (a dict: source, page, row…, used later for filtering and citations).

Method	Behaviour	Use when
`.load()`	Loads all documents at once into a list	Most cases
`.lazy_load()`	Yields documents one at a time (a generator)	Very large datasets that would not fit in memory

```
for doc in loader.lazy_load():
    process(doc)
```

Common loaders: `TextLoader`, `CSVLoader`, `UnstructuredHTMLLoader`, `WebBaseLoader` (needs BeautifulSoup), `JSONLoader` (needs a `jq_schema`), `PyPDFLoader`, `DirectoryLoader` (with `glob="*.md"`, `"*.pdf"`, …), YouTube transcript/audio loaders. LangChain lists 100+ integrations; look up the one you need rather than memorising them.

2. Chunking: what and why

Chunking (document splitting / transformation) is breaking a large document into smaller, manageable pieces called **chunks**. The lecture called it "50% of RAG performance".

Analogy: a maths book split into chapters (addition, subtraction, multiplication…). A multiplication question retrieves only the multiplication chapter, not the whole book. A good chunk holds **one clear idea**, not five unrelated ones.

Why we chunk (interview answer)

1. **Stay within limits** – embedding models and LLMs have a maximum input size; an oversized document cannot be processed in one go.
2. **Avoid "lost in the middle"** – LLMs use information at the start and end of a long context better than information buried in the middle.
3. **Retrieve only what's relevant** – the retriever returns a few relevant chunks instead of whole documents, which gives:
   - **Efficiency** – less text to process, lower latency
   - **Lower cost** – fewer tokens per request
   - **Better answer quality** – less noise, fewer hallucinations

Without chunking, a 1,000-page PDF may exceed the token limit, retrieval becomes slow, and important information gets lost.

The key trade-off: recall vs. precision

Chunk size	Recall	Precision	Effect
Large chunks	High	Low	Answer is likely somewhere in the chunk, but with lots of irrelevant text
Small chunks	Low	High	Very focused, but context can be cut off

Chunk size guidelines (from class)

Size	Tokens	Good for
Small	200–500	Specific factual questions, less noise
Medium	500–1,000	General Q&A, balanced (the usual default)
Large	1,000+	Complex reasoning that needs more context

There is no fixed rule; tune by evaluating retrieval quality.

Chunk overlap

Overlap = how much of the end of one chunk is repeated at the start of the next. It prevents a sentence or name from being cut in half and losing meaning (e.g. "My name is Sund | aram").

- Recommended: **10–20% of chunk size** (e.g. `chunk_size=1000, chunk_overlap=100-200`). Don't use 0 in production.
- More overlap → better context continuity but more chunks (more storage, slightly slower).

3. The five chunking strategies

Document-structure chunking is the usual default (cheap and accurate on PDFs and policies). LLM-based chunking gives the best accuracy at the highest cost.

Strategy	How it splits	Speed	Cost	Accuracy	Best use case
1. Length-based	Fixed number of characters/tokens (e.g. every 500)	Very fast	Free	Low–medium	Plain text, transcripts, logs, quick POCs
2. Text-structure-based	Paragraphs → lines → words (recursive separators)	Fast	Free	Medium–high	Articles, blogs, research papers
3. Document-structure-based	Headings, sections, chapters, tables, HTML/Markdown tags, code blocks	Fast	Free	High	PDFs, policies, SOPs, regulatory docs, Markdown/HTML
4. Semantic	Embeds sentences; starts a new chunk where meaning (topic) shifts	Medium	Low (embedding calls)	Very high	Mixed knowledge bases

Strategy	How it splits	Speed	Cost	Accuracy	Best use case
5. LLM-based	An LLM reads the text and picks natural topic boundaries (can also summarise each chunk)	Slow	High (LLM calls)	Best	Domain-specific data: legal, finance, healthcare

Notes on each

- **Length-based:** simple, fast, works on unstructured text. Downside: can cut sentences in the middle, lowering retrieval quality.
- **Text-structure-based:** keeps sentences and paragraphs whole, so meaning is preserved. Popular in industry.
- **Document-structure-based:** respects how the author organised the document (heading, sub-heading, footnote, table, chapter). Great for HR policies (leave, travel, reimbursement, security).
- **Semantic:** topic-aware, with high retrieval accuracy and good context preservation. Chunks can be inconsistent in size.
- **LLM-based:** the model "understands" content and creates intelligent chunks. Can be run on a local LLM (e.g. via Ollama) to avoid API cost.

Which to choose

- Beginner / no budget → length, text or document-structure.
- Most RAG apps (PDFs, policies) → **document-structure** (low cost, high accuracy).
- Accuracy matters more than cost, or domain-specific data → **LLM-based** (warn the client about latency).
- Middle ground → **semantic**.
- Managed cloud (e.g. Azure AI Search) can handle chunking and indexing for you.

Interview tip: "My RAG answers are poor or slow – what do you check first?" → the chunking strategy, then chunk size/overlap, then embedding dimensions.

4. Chunking code in LangChain

```
pip install -U langchain-text-splitters langchain-experimental langchain-openai tiktoken
```

Separator cheat-sheet

Separator	Splits by
`"\n\n"`	Paragraph
`"\n"`	Line
`" "`	Word
`""`	Character

1. Length-based: CharacterTextSplitter

```
from langchain_text_splitters import CharacterTextSplitter

splitter = CharacterTextSplitter(
    separator="",          # split anywhere (pure length)
    chunk_size=50,         # measured by length_function
    chunk_overlap=5,
    length_function=len,   # len = characters
)
chunks = splitter.split_text(text)
print(len(chunks), chunks[:3])
```

In class, 19 chunks of about 49–50 characters each (whitespace is stripped, so some are 49). Adding `chunk_overlap=5` repeats the last 5 characters at the start of the next chunk.

Print chunks in colour to see the boundaries:

```
from termcolor import colored
from random import choice

colors = ["red", "green", "yellow", "blue", "magenta", "cyan"]

def display_chunks(chunks):
    print(f"Total chunks: {len(chunks)}\n")
    for i, chunk in enumerate(chunks, start=1):
        print(f"Chunk {i} ({len(chunk)} chars)")
        print(colored(chunk, choice(colors)), end="\n\n")
```

Token-based: measure by tokens instead of characters

```
splitter = CharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base",   # or model_name="gpt-4"
    chunk_size=20, chunk_overlap=0,
)
```

`tiktoken` is OpenAI's BPE (byte-pair encoding) tokenizer. Try tokenisation interactively in the OpenAI tokenizer page.

2. Text-structure-based: RecursiveCharacterTextSplitter (most common)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    separators=["\n\n", "\n", " ", ""],  # paragraph -> line -> word -> char
    chunk_size=1000,
    chunk_overlap=100,
)
chunks = splitter.split_documents(docs)   # keeps metadata
# splitter.split_text(text) for raw strings
```

It tries the biggest separator first and only falls back to smaller ones when a piece is still too large. `RecursiveCharacterTextSplitter.from_tiktoken_encoder(...)` gives the token-measured version.

3. Document-structure-based

```
from langchain_text_splitters import (
    MarkdownHeaderTextSplitter, HTMLHeaderTextSplitter,
    RecursiveJsonSplitter, Language,
)

# Markdown – split on headings, heading text goes into metadata
md = MarkdownHeaderTextSplitter(headers_to_split_on=[
    ("#", "Header 1"), ("##", "Header 2"), ("###", "Header 3")])
md_chunks = md.split_text(markdown_text)

# HTML – split on h1/h2/h3
html = HTMLHeaderTextSplitter(headers_to_split_on=[
    ("h1", "Header 1"), ("h2", "Header 2"), ("h3", "Header 3")])
html_chunks = html.split_text(html_text)

# JSON – keeps nested structure intact
json_chunks = RecursiveJsonSplitter(max_chunk_size=300).split_json(json_data)
```

```
# Code – splits on functions/classes
py = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON, chunk_size=50, chunk_overlap=10)
code_chunks = py.split_text(python_code)
```

4. Semantic chunking

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

splitter = SemanticChunker(
    OpenAIEmbeddings(model="text-embedding-3-small"),
    breakpoint_threshold_type="percentile",  # or "standard_deviation",
"interquartile", "gradient"
)
chunks = splitter.split_text(text)   # in class: 4 chunks of unequal length
```

It embeds each sentence, measures the distance between neighbours, and breaks where the distance exceeds a statistical threshold. Set the API key first:

```
import os, getpass
os.environ["OPENAI_API_KEY"] = getpass.getpass("OpenAI API key: ")
```

5. LLM-based chunking (structured output)

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

class Chunk(BaseModel):
    chunk_text: str
    summary: str

class Chunks(BaseModel):
    chunks: list[Chunk]

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
llm_chunker = llm.with_structured_output(Chunks)

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert text chunker. Split the given text at
natural topic "
               "boundaries. Do not change the existing text. For each chunk,
also write "
```

```
                "a one- or two-line summary."),
    ("human", "Split the given text into chunks:\n\n{text}"),
])

chain = prompt | llm_chunker          # LCEL: prompt piped into model
result = chain.invoke({"text": text})
for c in result.chunks:
    print(c.chunk_text, "->", c.summary)
```

In class this returned 3 chunks, each ending on a full sentence. Tips from class: prefer `ChatOpenAI` over `OpenAI` and `ChatPromptTemplate` over `PromptTemplate`.

5. Embeddings (intro)

Embedding = converting text into a vector of numbers that captures its meaning, so a computer can compare texts mathematically. The maths (how vectors are produced and compared) is covered in the next session.

Short history

Year	Model	By
2013	Word2Vec (CBOW, Skip-gram)	Google
2014	GloVe	Stanford
2016	FastText	Facebook (Meta)
2017	Transformer ("Attention Is All You Need") – foundation of GenAI	Google
2018	BERT (encoder, "fill in the blank") and GPT (decoder, generation)	Google / OpenAI
Today	Dedicated embedding APIs, e.g. OpenAI `text-embedding-3`	OpenAI and others

OpenAI embedding models

Model	Default dimensions	Notes
`text-embedding-3-small`	1,536	Recommended default: cheap, efficient
`text-embedding-3-large`	3,072	Higher quality, several times the price

Model	Default dimensions	Notes
`text-embedding-ada-002`	1,536	Older generation; avoid for new work

Both `-3` models accept up to about 8K tokens per input, which is another reason to chunk. Check current pricing on OpenAI's site; embedding calls are billed per token.

Code: embed one text, then a whole PDF

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings

emb_small = OpenAIEmbeddings(model="text-embedding-3-small")
emb_large = OpenAIEmbeddings(model="text-embedding-3-large")

# One query/text -> one vector
vec = emb_small.embed_query("RAG grounds LLM answers in your own data.")
len(vec)                      # 1536 (large -> 3072)

# Step 1: load
docs = PyPDFLoader(r"data/pdf/attention_is_all_you_need.pdf").load()   # 15
pages

# Step 2: chunk
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)                                 # 93
chunks

# Step 3: embed (many texts -> list of vectors)
texts = [c.page_content for c in chunks]
vectors = emb_small.embed_documents(texts)   # 93 vectors x 1536 numbers
```

Method	Input	Output
`embed_query(text)`	One string	One vector
`embed_documents(texts)`	List of strings	List of vectors (one per chunk)

Bug hit in class: the result was stored in a variable with the same name as the embedding model (`embedding_small = embedding_small.embed_query(...)`). That overwrote the model with a list, so `.embed_documents` no longer existed. Use different names for the model and its output.

Reducing dimensions

```
emb = OpenAIEmbeddings(model="text-embedding-3-small", dimensions=256)   # or 512
```

- Smaller vectors → less storage, faster similarity search, lower latency.
- Small accuracy drop (the lecture's example: ~96% → ~88%). Higher dimensions generally give better answers.
- Always balance **cost** and **time**: customers won't wait minutes for an answer.

Latency checklist so far: (1) better chunking, (2) reduce embedding dimensions. More techniques come later (caching, re-ranking, etc.).

6. Q&A highlights

Question	Answer
How do I pick chunk size?	Small 200–500, medium 500–1,000, large 1,000+ tokens depending on the data; ~1,000 with 10–20% overlap is a common start. Tune with evaluation.
Is chunking based on cosine similarity?	No. Basic chunking just splits text. Cosine similarity is used later in retrieval (and inside semantic chunking).
7 chunks are relevant but k = 5 – is the answer incomplete?	The retriever ranks chunks by similarity score and returns the 5 best. Usually enough; if not, increase k or use re-ranking/hybrid search (later).
Does chunking remove duplicate text?	No. Chunking only splits. Deduplicate during preprocessing, or tell the LLM in the prompt to ignore repeats.
What is overlap for?	Repeats the tail of one chunk at the start of the next so meaning isn't cut off. Use 10–20%; 0 is faster but less accurate.
PDF with images?	`PyPDFLoader` reads text only. Images/scans need OCR or multimodal loaders (covered in multimodal RAG).
Can I use a local LLM for LLM-based chunking?	Yes, e.g. a model served by Ollama, avoiding API costs.

Question	Answer
Semantic vs. LLM-based – what's the difference?	Semantic uses embeddings plus a statistical threshold (cheap). LLM-based has a model read the text and decide (expensive, best for domain-specific data).
Can I build RAG over a code base (MD, PDF, Python)?	Yes. Use language-aware splitters for code and header splitters for Markdown.
Is metadata kept during chunking?	Yes, with `split_documents()`; each chunk inherits its document's metadata.
Loading from software that needs a login?	Use that system's API with credentials, or a dedicated connector; generic loaders don't handle authentication.
Hugging Face vs. LangChain?	Hugging Face = a platform/community hosting open-source models (like GitHub for models). LangChain = a framework that wires models, loaders, splitters and vector stores together.
Does reducing dimensions lose context?	The whole chunk is still encoded, just more compactly. Expect a small accuracy drop in exchange for speed and lower storage.

7. Expert corrections to the lecture

Several points in this session were imprecise. Use these versions in interviews.

1. **"Lost in the middle" is not truncation.** It is a finding (Liu et al., 2023) that LLMs given a long context use information at the beginning and end better than in the middle. It is not that the model "reads the first 2,500 and last 2,500 tokens and forgets the rest". Exceeding the context limit is a separate problem: the input is rejected or truncated.
2. **`chunk_size` units:** with `length_function=len`, sizes are **characters**, not words or "vocabulary". Use `from_tiktoken_encoder` to measure tokens.
3. **Separators:** `" "` splits by word and `""` by character. An underscore is not a word separator. `CharacterTextSplitter` with `separator=" "` never cuts a word in half; mid-word cuts ("Sund|aram") only happen with `separator=""`.
4. **Tokenizer name:** it is `cl100k_base` (GPT-4 / GPT-3.5); newer GPT-4o models use `o200k_base`.

5. **Semantic chunking is not tiktoken and not an LLM.** It uses an *embedding model* plus a statistical breakpoint rule (percentile, standard deviation, interquartile, gradient). Its cost is only embedding calls, which are cheap. `breakpoint_threshold_amount=0.1` is unusual; the percentile default is 95.
6. **Retrieval ranks by similarity score, not "probability".** Top-k = the k chunks with the highest similarity (e.g. cosine) to the query.
7. **Model limits were mixed up:** BERT accepts max **512 tokens**; its hidden size is **768 (base) / 1,024 (large)**. GPT-3 had a 2,048-token context. OpenAI `text-embedding-3` models accept ~8K tokens. Modern chat LLMs handle 128K–1M+ tokens, so we chunk mainly for retrieval precision, cost and latency.
8. **One vector per chunk** (answering a class question): the model produces a vector per token internally, then pools them into a single fixed-length vector for the whole chunk.
9. **Reducing `dimensions` doesn't lower the embedding API bill** (billed per input token). It reduces vector storage and search time.
10. **The "system hang" wasn't a CPU vs. GPU issue.** OpenAI embeddings run on OpenAI's servers. The slowdown came from the API calls and printing a huge vector list. A GPU matters only for local models (e.g. Hugging Face).
11. **Temperature error with GPT-5 mini:** reasoning models only accept the default temperature, and `ChatOpenAI` sent 0.7. Switching to `gpt-4o-mini` worked; alternatively, don't set temperature.
12. **Method name:** it is `with_structured_output` (not "with_structure_output"); the import is `from langchain_experimental.text_splitter import SemanticChunker`.
13. **DuckDuckGo** is a web-search tool, not an authenticated data loader.
14. **GloVe** was published in 2014 (the lecture said 2015).

8. Interview quick revision and what's next

60-second answer: "Explain chunking"

> Chunking splits loaded documents into smaller pieces so they fit model limits, avoid the lost-in-the-middle effect, and let the retriever return only relevant text. That improves latency, cost and answer quality. The trade-off is recall vs. precision: big chunks have high recall and low precision, small chunks the reverse. I set `chunk_size` around 500–1,000 tokens with 10–20% overlap. There are five strategies: length-based, text-structure (recursive), document-structure (headers/sections), semantic (embedding breakpoints) and LLM-based. I default to document-structure for PDFs and policies, and use LLM-based when accuracy on domain data matters more than cost.

30-second answer: "What is an embedding?"

> An embedding turns text into a dense vector that captures meaning, so similar texts get nearby vectors. In RAG we embed every chunk once and store the vectors in a vector DB, then embed the user's query with the same model and find the nearest chunks. I usually use `text-embedding-3-small` (1,536 dims), reducing dimensions to cut latency and storage.

What's next

1. **Embedding theory** (maths behind vectors and similarity) – Saturday.
2. **End-to-end simple RAG:** vector DB, retrieval techniques, augmentation, generation, one real-time case study.
3. **Advanced RAG** (around 20–21 June), then agentic AI: AutoGen, CrewAI, LangGraph, MCP, guardrails, observability (LangSmith), fine-tuning, Hugging Face and Ollama.

Homework

- [ ] Re-run every splitter above on your own text and compare chunk counts
- [ ] Try `chunk_overlap` of 0 vs. 10% vs. 20% and inspect the boundaries
- [ ] Embed a PDF with `dimensions=256` and compare speed with the default
- [ ] Explain chunking to someone in your own language (Feynman technique)
- [ ] Explore the chunk visualiser link shared in class (character / token / sentence / recursive splitters)

SESSION 3

Saturday, 13 June 2026

Building an End-to-End RAG Chain

RAG Notes – Session 3 (13 June 2026)

Oct 14, 2026 · @Vljay

1. Recap

This session builds the first **end-to-end traditional (simple) RAG system** in code. Session 1 covered loading, and Session 2 covered chunking and embedding.

How we got here: three eras of AI

Era	Approx. years	Idea	Limitation / breakthrough
Rule-based AI	1950s–2000	Hand-written if/else rules, domain-specific logic	Rigid; poor scalability and adaptability
Machine learning	1990s–2012	Data-driven: learn patterns/functions from data (X → Y)	Struggled with very large, unstructured data
Deep learning	2012–2020	Neural networks (MLP, CNN, NLP models) on big data	Led to the Transformer (2017)
Generative AI	2017–present	Transformer → BERT/GPT (2018) → ChatGPT (2022): generate text, images, code; reasoning, agents	Today's focus

Course so far: Python → ML → prompt engineering (CoT, ReAct, few/zero-shot, personas) → LangChain (simple no-RAG chatbot) → RAG.

The lecture's advice: use vibe coding, but keep your own coding skills so you can debug.

RAG architecture, with the retrieval detail added

1. **Indexing:** source data → load (LangChain or LlamaIndex) → chunk → embed → store in a vector DB.
2. **Retrieval:** embed the user query → search the vector DB → return the top-k chunks (the **context**).
3. **Augmentation:** prompt = instructions + context + query.
4. **Generation:** the LLM answers only from the context → response to the user.

Bi-encoder vs. cross-encoder

	Bi-encoder	Cross-encoder
How it works	Encodes the query and each document **separately** into vectors, then compares them (e.g. cosine)	Feeds the query and one document **together** into a Transformer, which outputs a relevance score
Speed	Fast: document vectors are precomputed	Slow: one model pass per (query, document) pair
Accuracy	Good	Higher
Role in RAG	**Stage 1 retrieval** over millions of chunks → ~100 candidates	**Stage 2 re-ranking** of those candidates → best top-k

Typical production flow: bi-encoder retrieval (with an HNSW index) → cross-encoder re-ranking → keep the top k (e.g. 5) as context. Today's code uses stage 1 only; re-ranking is covered later.

2. The 7-step mantra for an end-to-end RAG chain

These seven steps are the answer to "Have you built a real RAG pipeline? Walk me through it."

1. **Initialise the vector DB connection** (here Chroma). This implies the indexing work is done: load → chunk → embed → store. Alternatives: Pinecone, Weaviate, Qdrant, Milvus, FAISS.
2. **Create a retriever object**, so the user query hits the knowledge base first, not the LLM directly.
3. **Initialise a chat prompt template**, the augmentation: system persona + rules + `{context}` + `{question}`.
4. **Initialise the generator**, i.e. the chat model (any LLM: OpenAI, Gemini, Groq, etc.).
5. **Initialise an output parser**, to shape the response the way stakeholders want (string, JSON…).
6. **Define the RAG chain**: retriever → prompt → LLM → parser.
7. **Invoke the chain** and inspect both what was **retrieved** and what the **LLM generated**. Evaluation (grounding, etc.) is covered later.

When is Chroma a good choice? For local or prototype work, storing vectors on your own disk. For hundreds of GB or more, or for heavy production use, pick a managed or scalable vector DB.

3. Code part 1: setup and indexing

Dataset: a zip of 4 research-paper PDFs (Attention, CNN, ResNet, Vision Transformer) plus one JSON-lines file of Wikipedia paragraphs.

Install (pin stable versions)

```
!pip install langchain langchain-openai langchain-community langchain-chroma
jq pymupdf
```

The lecture recommends pinning older, well-tested versions over a release from yesterday. That's what "stable" means. Install large packages in separate cells, and restart the runtime if Colab asks.

API key

```
import os
from getpass import getpass
os.environ["OPENAI_API_KEY"] = getpass("OpenAI API key: ")
```

In Colab you can also store keys in the **Secrets** (key icon) panel. In VS Code, use a `.env` file.

Embedding model

```
from langchain_openai import OpenAIEmbeddings
embedding_model = OpenAIEmbeddings(model="text-embedding-3-small")
```

Get the data

```
!unzip -q rag_docs.zip      # creates rag_docs/ with 4 PDFs + 1 JSON file
```

Load the JSON and move ID/title into metadata

Each line looks like `{"id": ..., "title": ..., "paragraph": ...}`. The ID and title are metadata; only the paragraph is real content.

```
import json
from langchain_community.document_loaders import JSONLoader
from langchain_core.documents import Document

loader = JSONLoader(
    file_path="rag_docs/wikidata_rag_demo.jsonl",
    jq_schema=".",
    text_content=False,
    json_lines=True,
)
wiki_docs = loader.load()          # 1,801 documents

wiki_docs_processed = []
for doc in wiki_docs:
    rec = json.loads(doc.page_content)
    metadata = {"title": rec["title"], "id": rec["id"], "source":
"Wikipedia"}
    p = rec["paragraph"]
    data = " ".join(p) if isinstance(p, list) else p
    wiki_docs_processed.append(Document(page_content=data,
metadata=metadata))
```

No chunking is needed here: every paragraph is short (~200–250 words), so splitting would just waste time.

Load and chunk all PDFs

```
from glob import glob
from langchain_community.document_loaders import PyMuPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

def create_simple_chunks(file_path, chunk_size=500, chunk_overlap=40):
    print("Loading pages:", file_path)
    pages = PyMuPDFLoader(file_path).load()
    print("Chunking pages:", file_path)
    splitter = RecursiveCharacterTextSplitter(chunk_size=chunk_size,
chunk_overlap=chunk_overlap)
    chunks = splitter.split_documents(pages)
    print("Finished:", file_path)
    return chunks

paper_docs = []
for fp in glob("rag_docs/*.pdf"):
    paper_docs.extend(create_simple_chunks(fp))   # extend, not append
len(paper_docs)     # 444 chunks
```

Use **`extend`** because each call returns a *list* of chunks: `append` would nest lists inside the list. PDF chunks keep rich metadata automatically (source, file path, page, total pages).

Combine and index into Chroma (run once)

```
total_docs = wiki_docs_processed + paper_docs    # 1,801 + 444 = 2,245

from langchain_chroma import Chroma

chroma_db = Chroma.from_documents(
    documents=total_docs,
    embedding=embedding_model,
    collection_name="my_db",
    persist_directory="./my_db",
    collection_metadata={"hnsw:space": "cosine"},  # HNSW index, cosine
distance
)
```

Run indexing only once. Every re-run adds the same 2,245 documents again: duplicates waste embedding money and fill the top-k with copies. To rebuild, delete `./my_db` first, or pass stable `ids=` so re-runs overwrite instead of duplicating.

Reload later from disk (safe to repeat)

```
chroma_db = Chroma(
    persist_directory="./my_db",
    collection_name="my_db",           # must match the name used when
indexing
    embedding_function=embedding_model,
)
```

4. Code part 2: retriever, prompt, chain, invoke

Step 2 – Retriever (top-5 by cosine similarity)

```
similarity_retriever = chroma_db.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 5},
)

docs = similarity_retriever.invoke("What is self-attention?")
for d in docs:
    print(d.metadata)
    print(d.page_content[:300], "\n")
```

The user's query is embedded with the same model, HNSW finds the nearest chunk vectors, and the top 5 become the context.

Step 3 – Augmentation prompt

```
from langchain_core.prompts import ChatPromptTemplate

rag_prompt = """You are an assistant who is an expert in question-answering
tasks.
Answer the following question using only the following pieces of retrieved
context.
If the answer is not in the context, do not make up answers; just say that
you don't know.
Keep the answer detailed and well-formatted based on the information from the
context.

Question:
{question}

Context:
{context}

Answer:"""

rag_prompt_template = ChatPromptTemplate.from_template(rag_prompt)
```

Step 4 – Generator

```
from langchain_openai import ChatOpenAI
chatgpt = ChatOpenAI(model="gpt-4o-mini", temperature=0)   # 0 = focused,
less creative
```

Steps 5 & 6 – Output parser and RAG chain (LCEL)

```
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

qa_rag_chain = (
    {
        "context": similarity_retriever | format_docs,   # R: retrieve + join
chunks
        "question": RunnablePassthrough(),              # pass the user query
through
    }
    | rag_prompt_template                               # A: build the
augmented prompt
    | chatgpt                                           # G: generate
    | StrOutputParser()                                 # output as plain
string
)
```

The `|` (pipe) chains components so each one's output feeds the next. `RunnablePassthrough()` forwards the original question unchanged, while the same question also drives the retriever.

Step 7 – Invoke

```
from IPython.display import display, Markdown

result = qa_rag_chain.invoke("What is machine learning?")
display(Markdown(result))
```

Without `StrOutputParser`, the result is an `AIMessage`; use `result.content`.

5. What the demo showed

The retriever always returns k chunks, even for irrelevant questions. The augmentation prompt is what stops a wrong answer.

Question asked	Retriever returned	Final RAG answer
What is self-attention?	5 chunks, mostly from the Attention paper	Grounded explanation
What is CNN?	5 chunks from the CNN paper	Grounded explanation
What is CNN? Explain simply with examples	Relevant CNN chunks	Simple, detailed answer, rephrased by the LLM but staying within the documents
"The nation wants to know what is happening with TMC party in Bengal"	5 loosely related Wikipedia chunks (they mention Bengal/Kolkata)	"I don't know"
Who is Narendra Modi in India?	Unrelated chunks	"I don't know"

Lessons

1. **Retrieval doesn't judge relevance.** It ranks by similarity and returns the closest k, even if all are weak matches.
2. **Augmentation is the guardrail.** "Answer only from the context, else say I don't know" turns weak context into an honest refusal.
3. **The knowledge base defines the bot's scope.** These docs are all ML/DL papers, so political questions are out of scope by design.

Filtering weak matches with a score threshold

```
retriever = chroma_db.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"score_threshold": 0.5, "k": 5},
)
```

It returns only chunks above the threshold, possibly none. Tune the value on your own data.

Adding citations

Add to the prompt: "Also state the source of the answer." Then pass each chunk's metadata (title/source/page) into the context. Richer metadata gives better citations (e.g. page numbers for PDFs).

Not yet covered (coming later)

- **Conversation memory** for follow-up questions ("What is *his* employee ID?")
- Hybrid search (BM25 + vectors), re-ranking, RAG-Fusion
- Guardrails, access control, observability
- Scheduled re-indexing when source data changes (daily/weekly, decided by the business; usually set up in the cloud)

6. Model types by input and output

Pick the model family from the problem's input → output.

Model type	Input → Output	Examples	Typical use
Language model (LLM)	Text → text	GPT-4o mini, Llama, Claude	Simple RAG, summaries, emails, Q&A
Reasoning model	Text/structured data → decisions, plans	OpenAI o-series, DeepSeek-R1	Multi-step reasoning, agents taking actions
Speech-to-text (ASR)	Audio → text	Whisper	Transcription
Text-to-speech (TTS)	Text → audio	OpenAI TTS, Amazon Polly	Reading content aloud
Speech-to-speech translation	Audio → audio	Meta SeamlessM4T	Spoken English → spoken Spanish
Text-to-image	Text → image	DALL·E, Imagen, Stable Diffusion, Flux	Generating visuals
Text-to-video	Text → video	OpenAI Sora	Short ad clips from a script

Model type	Input → Output	Examples	Typical use
Multimodal	Any mix → any mix	GPT-4o ("omni"), Gemini 2.5 Pro	Images + text in one conversation; basis of multimodal RAG

Today's simple RAG uses only a text → text LLM. Multimodal RAG (images, tables) comes later.

7. Q&A highlights

Question	Answer
Why weren't ID/title in metadata automatically for JSON?	`JSONLoader` put the whole record in `page_content`; we moved ID/title into metadata by hand. PDFs already come with rich metadata. It depends on the data.
Is the LLM only for generation in RAG?	No. An embedding model also converts text to vectors (at indexing and for every query). The chat LLM then generates. Both consume tokens.
Can one `DirectoryLoader` handle mixed JSON + PDF?	Yes, with a helper that picks the loader per extension; the lecture split them to show custom metadata handling.
Who builds the pipeline in a company?	Usually one engineer owns the RAG backend end to end; CI/CD and deployment may be another team.
Can we mix embedding providers?	The **same embedding model** must be used for indexing and for queries; vectors from different models aren't comparable. The chat LLM can come from a different provider.
I ran indexing twice – can I undo?	No undo. Delete the collection/directory and rebuild (paying for embeddings again).
Can I load an Excel workbook with several sheets?	Yes, e.g. with `openpyxl`/pandas, reading each sheet into its own documents.
Retrieved chunks overlap each other – a problem?	Overlap means neighbouring chunks share text. That's expected; tune overlap or switch chunking strategy if it hurts.

Question	Answer
What does a RAG request cost?	Indexing embeddings (one-off), the query embedding, vector DB hosting/search, and LLM input + output tokens.
How often to re-index?	Business-driven: hourly to monthly, via a scheduler (cloud-based in production).
Can examples in the answer be hallucinated?	With the "only from context" rule, the LLM rephrases and explains using the retrieved material; it should not pull in outside facts.
Can different users see different answers?	Yes: role-based access (filter retrieval by user permissions), or route to different agents/personas per department (agentic router, later).
Can confidential data be kept out of answers?	Yes, via access control, metadata filters and guardrails (covered later).

8. Expert corrections to the lecture

1. **Cross-encoder ≠ "merging two documents".** A cross-encoder scores one *(query, document)* pair at a time by reading both together.
2. **Today's code had no cross-encoder.** `as_retriever(search_type="similarity", k=5)` is bi-encoder retrieval over an HNSW index. Re-ranking needs an explicit step, e.g. `CrossEncoderReranker` with a model like `BAAI/bge-reranker`.
3. **Retrieval scores are similarity/distance values, not probabilities.** Top-k simply means the k most similar chunks.
4. **HNSW = Hierarchical Navigable Small World**, an approximate nearest-neighbour graph index. It makes search fast; it is not an embedding algorithm.
5. **Duplicates aren't detected by RLHF.** RLHF is a technique for *training* models with human preferences. To avoid or catch duplicate indexing: check `chroma_db._collection.count()`, use deterministic `ids=`, or clear the directory before re-indexing.
6. **"Supervised self-learning" isn't a standard term.** The Q&A-pair discussion maps to *supervised fine-tuning*. A RAG system involves no training at all.
7. **No GPU needed for this notebook.** Embeddings and generation run on OpenAI's servers; a GPU matters only for local models.

8. **The Chroma reload error** came from the collection name. `from_documents` without a name uses the default `"langchain"`, so reload with the *same* `collection_name` (or omit it in both places).
9. **Use current imports:** `langchain_community.document_loaders` and `langchain_core.documents`, not the deprecated `langchain.document_loaders` / `langchain.docstore`. Parse a JSON string with `json.loads`, not `json.load`.
10. **`temperature=0` reduces randomness,** but grounding plus the "only from context" rule is what reduces hallucination.
11. **Similarity cut-offs use `score_threshold`** (`search_type="similarity_score_threshold"`). `top_p` is a generation parameter, not a retrieval one.
12. **Whisper is speech-to-text,** not text-to-speech; TTS examples are OpenAI TTS or Amazon Polly.
13. **Retrieval search itself isn't billed per token.** Costs are embeddings, LLM tokens and vector DB hosting. The "$3,000 minimum quote" was the lecturer's rough figure, not a standard.
14. **Follow-up questions need conversation memory** (chat history fed into the prompt or a history-aware retriever), not a cache.

9. Interview quick revision and homework

60-second answer: "Describe a RAG system you built"

> I built a Q&A assistant over internal documents: PDFs and a JSON knowledge export. I loaded them with LangChain loaders, cleaned the JSON so ID and title became metadata, and chunked the PDFs with a recursive splitter (500 characters, 40 overlap). Short JSON records weren't chunked. I embedded everything with `text-embedding-3-small` and stored ~2,200 chunks in Chroma with an HNSW cosine index. At query time a retriever returns the top 5 chunks. An augmentation prompt tells the LLM to answer only from that context or say "I don't know". `gpt-4o-mini` at temperature 0 generates the answer, wired as an LCEL chain: retriever → prompt → LLM → parser. Next steps were re-ranking, citations, memory and evaluation.

Key one-liners

- Bi-encoder = fast first-stage retrieval; cross-encoder = accurate second-stage re-ranking.

- The retriever always returns k chunks; the prompt decides whether they're good enough.
- Index once; duplicates break retrieval quality and cost money.
- Same embedding model for indexing and querying.

Homework

- [ ] Rebuild this notebook on a different dataset of your own
- [ ] Ask in-scope and out-of-scope questions and check for "I don't know"
- [ ] Add a citation instruction and print each chunk's metadata
- [ ] Try `similarity_score_threshold` and compare results
- [ ] Draw the 7-step mantra from memory

SESSION 4

Sunday, 14 June 2026

Embedding Theory & Retrieval Methods

RAG Notes – Session 4 (14 June 2026)

Oct 14, 2026 · @VIjay

1. What an embedding is

An embedding converts something (text, image, audio, video, code, a document) into a vector of numbers that captures its meaning. Algorithms only work with numbers, so embeddings are the bridge between human content and AI processing.

> Human language → numbers (embedding) → AI processing → meaningful output

Intuition 1: words as numbers

Word	Embedding (toy, 3 dims)
dog	[0.90, 0.30, 0.20, …]
puppy	[0.88, 0.33, 0.22, …]
car	[-0.40, 0.70, 1.20, …]

"dog" and "puppy" get nearly identical numbers because they mean similar things; "car" is far away.

Intuition 2: describing a person with features

A person can be described by features: height 170, weight 75 kg, age 35, energy 8/10, BMI 24 → **[170, 75, 35, 8, 24]**. An embedding does the same, except the model **learns** the features itself ("animal-ness", "pet-ness", "vehicle-ness"…). Those learned features live in a neural network's hidden layers, so humans can't name what each number means. That's why embeddings feel like a black box.

	Machine learning	Deep learning (embeddings)
Features	Chosen manually; visible (e.g. feature importance in a random forest)	Learned automatically in hidden layers; not interpretable
Control	You control which features are used	You don't control individual features

Where embeddings are used

Semantic search, RAG retrieval, recommendations, machine translation, clustering similar items, duplicate-question detection, resume ↔ job-description matching, similar-code search.

2. From one-hot to Transformers

Older methods treat every word as unrelated to every other word. Dense learned embeddings let similar meanings land close together, which is what RAG needs.

Why keyword matching fails

Question: "How can I **reset** my **laptop password**?" Document: "Steps to **recover device login credential**".

A human sees reset≈recover, laptop≈device, password≈credential. Keyword-based methods see zero overlap, so the right document is missed. Embeddings fix this by capturing **semantic** (similar-meaning) relationships.

Three generations of NLP representation

Generation	Techniques	Year / by	Captures meaning?
Frequency-based (ML era)	One-hot, Bag of Words, TF-IDF, n-grams	Classic NLP	No: sparse, mostly zeros
Distributed (static) embeddings	Word2Vec (CBOW, Skip-gram), GloVe, FastText	2013 Google, 2014 Stanford, 2016 Facebook	Yes, one fixed vector per word
Contextual embeddings (LLM era)	Transformer → BERT, GPT, modern embedding APIs	2017 Google, 2018 Google/OpenAI	Yes, the vector depends on context ("bank" of a river vs. a bank)

Worked example: three reviews

R1 "I like food" · R2 "I do not like food" · R3 "Food is awesome and cheap" → vocabulary of 9 words: *I, like, food, do, not, is, awesome, and, cheap*.

Method	How R1 is represented	Problem
One-hot	Each word is a 9-long vector with a single 1 ("I" = [1,0,0,0,0,0,0,0,0])	Vector length = vocabulary size (50,000+), extremely sparse, no meaning
Bag of Words	One vector per sentence with word counts: [1,1,1,0,0,0,0,0,0]	Still sparse; ignores word order; every word weighted equally
TF-IDF	Like BoW, but rare words get more weight than common ones ("awesome" > "food")	Still sparse, no semantics
n-grams (n = 2)	Pairs: "I like", "like food" vs. "do not", "not like"…	Keeps some order ("not like" ≠ "like"), but the vocabulary explodes

Stop words (I, a, the, is, do…) are often removed. Careful: removing "do not" makes R1 and R2 look identical, so build n-grams *before* removing stop words.

Dense embeddings

Word2Vec and later models represent each word or text with a fixed number of learned dimensions instead of one slot per vocabulary word. Humans can visualise 2–3 dimensions; models use hundreds or thousands, handled with linear algebra.

Model	Typical embedding dimensions
Word2Vec / GloVe	300 (configurable)
Original Transformer	512
BERT-base / BERT-large	768 / 1,024
OpenAI ada-002, text-embedding-3-small	1,536
OpenAI text-embedding-3-large	3,072

How text becomes an embedding

1. **Input text:** "I like mango"
2. **Tokenise:** split into tokens → token IDs
3. **Neural network (Transformer):** contextual vectors per token
4. **Pool** into one fixed-length vector: [0.14, 0.29, -0.77, …] (e.g. 1,536 numbers)

3. Measuring similarity

Once texts are vectors, "similar meaning" becomes "vectors close together". RAG most often uses **cosine similarity**.

Example: S1 "I want to buy a mobile phone" and S2 "I am looking for a smartphone" get nearby vectors. S3 "The server dataset crashed" lands far from both.

Toy 2-D vectors: apple = (2, 3), mango = (2.2, 3.1), car = (8, 9). Apple and mango are close; car is far.

The three common measures

Euclidean distance (straight-line distance; smaller = more similar):

```
d(A,B) = \sqrt{\sum_i (A_i - B_i)^2}
```

apple vs. mango: $\sqrt{(2-2.2)^2 + (3-3.1)^2} \approx 0.22$ · apple vs. car: $\sqrt{(2-8)^2 + (3-9)^2} \approx 8.49$

Cosine similarity (angle between vectors, ignoring length; larger = more similar):

```
\cos(\theta) = \frac{A \cdot B}{\lVert A \rVert \, \lVert B \rVert}
```

Angle	cos	Meaning
0°	1	Same direction: very similar meaning
90°	0	Unrelated
180°	−1	Opposite direction

Dot product (combines angle and length; used when vectors are normalised, which makes it equal to cosine):

```
A \cdot B = A_1 B_1 + A_2 B_2 + \dots + A_n B_n
```

Other distance metrics you may hear (from KNN): Manhattan, Minkowski, Hamming.

Where embeddings sit in the RAG flow

1. User asks a question.
2. The question is converted into an embedding (same model as the documents).
3. The vector DB is searched for the most similar chunk embeddings.
4. The most relevant chunks (the context) are retrieved.
5. Question + context are sent to the LLM (through the augmentation prompt).

6. The LLM generates a grounded answer.

Good embeddings → better retrieval → fewer hallucinations. But embeddings alone aren't magic: different models produce different vector spaces, and pure semantic search can miss exact terms (IDs, codes, names). Hence hybrid search, next.

4. Retrieval methods

Retrieval quality decides answer quality: if the context is wrong, augmentation and generation fail too. Production RAG usually runs **hybrid search (BM25 + vectors) followed by re-ranking**.

Method	What it does	Strength	Weakness	Use when
Keyword search (BM25)	Matches exact words; BM25 scores them by term frequency, rarity (IDF) and document length	Exact IDs, policy numbers, names, codes, jargon	Misses synonyms ("reset" vs "recover")	Exact-match lookups
Semantic / vector search	Embeds query and chunks; finds nearest vectors (cosine)	Understands meaning and paraphrases	Can miss exact terms, rare IDs	Chatbots, FAQs, natural-language questions
Hybrid search	Runs keyword + vector search and merges the results	Best of both	Slightly more complex	Default for enterprise RAG
Re-ranking	Re-scores the top ~50–100 candidates with a stronger model (e.g. cross-encoder) and reorders them	Much better precision in the top k	Extra latency/cost per query	When the top results are close but not the best
Reciprocal Rank Fusion (RRF)	Merges several ranked lists by rank position	Simple, no tuning, robust	Only as good as its input lists	Combining keyword + vector lists in hybrid search

Method	What it does	Strength	Weakness	Use when
RAG-Fusion	LLM rewrites the question into several variants, retrieves for each, merges with RRF	Better recall for vague questions	Extra LLM calls (cost, latency)	Hybrid isn't enough

Examples from class

- Keyword: "How to reset my password?" finds "Step 2: reset password" but misses "Recover account credential".
- Semantic: "Work from home policy" finds "Remote work guideline"; "Can I take a leave?" finds the vacation policy.
- Hybrid: a healthcare RAG must match exact patient IDs (keyword) *and* understand symptoms described in different words (semantic).

Re-ranking in two levels

- **L1:** fast retrieval: BM25 + vector similarity → candidate list.
- **L2 (semantic re-ranking):** a stronger model re-reads query + each candidate and reorders them → keep the top k.

RRF formula

```
\text{RRF}(d) = \sum_{r \in \text{lists}} \frac{1}{k + \text{rank}_r(d)}
```

k is a constant (commonly 60). A document ranked high in several lists rises to the top.

When answers are poor: improve retrieval before fine-tuning

1. Fix chunking (strategy, size, overlap).
2. Switch from pure vector search to hybrid (BM25 + vector).
3. Add re-ranking.
4. Try RAG-Fusion / query rewriting.
5. Only then consider fine-tuning, which is far more expensive.

5. Model parameters explained

Parameters are the learned weights and biases of a neural network. "70B parameters" means 70 billion such numbers. Parameter count is separate from a model's input limit and from its embedding size.

Worked example (small network)

Inputs: 5 features (height, weight, age, energy, BMI) → hidden layer 1 with 3 neurons → hidden layer 2 with 2 neurons → 1 output.

Connection	Weights	Biases	Total
Input (5) → Hidden 1 (3)	5 × 3 = 15	3	18
Hidden 1 (3) → Hidden 2 (2)	3 × 2 = 6	2	8
Hidden 2 (2) → Output (1)	2 × 1 = 2	1	3
Total			**29 parameters**

Each neuron computes a weighted sum + bias, then applies an activation function. Training adjusts the weights through forward propagation and **backpropagation** (popularised by Rumelhart, Hinton and Williams in 1986).

Four numbers people confuse

Term	Meaning	Example
Parameters	Learned weights + biases (model size)	GPT-3: 175B; DeepSeek V4 Pro: ~1.7T (per class)
Context window / max input	Max tokens the model accepts at once	OpenAI embedding models: ~8K tokens; chat LLMs: 128K–1M+
Embedding dimension	Length of the output vector	text-embedding-3-small: 1,536
Training corpus size	Tokens of training data	Tens of trillions of tokens for frontier models

The Wikipedia "Large language model" page lists parameter counts over time, e.g. GPT-1 0.117B (2018), BERT 0.34B, T5 11B, GPT-2 1.5B, GPT-3 175B (2020). Some vendors (e.g. Gemini) don't disclose theirs.

6. Multimodal RAG (preview for next session)

Multimodal RAG handles inputs (and optionally outputs) of mixed types: text, images, tables, audio, code, PDFs that combine all of these.

- Use a multimodal model, e.g. **GPT-4o** ("o" = omni = all modalities) or **Gemini 2.5 Pro**.
- PDFs with text + images + tables need specialised extraction (e.g. Unstructured.io), with a different technique per element type.
- Upcoming topics: Redis and doc stores, latency at scale (e.g. 10,000 users), Graph RAG with Neo4j, evaluation, guardrails, observability.

7. Q&A highlights

Question	Answer
If the query and document share the exact phrase, does semantic search still find it?	Yes. Identical text gives near-identical vectors; semantic search finds exact and similar wording.
How do bi-encoder/cross-encoder relate to these search methods?	Bi-encoder = the vector (semantic) search stage; cross-encoder = the re-ranking stage. Hybrid search + re-ranking is the full pipeline.
What do "8B/70B parameters" mean if input is limited to a few thousand tokens?	Parameters = model weights (capacity). The input limit is a separate design constraint, which is why we chunk.
Does RRF call an LLM to generate queries?	RRF itself doesn't; it just merges ranked lists. *RAG-Fusion* uses an LLM to generate query variants, then merges with RRF.
I upload files to ChatGPT/Copilot – is that RAG?	Products often retrieve from uploaded files internally (a managed RAG), but you don't control the chunking or search. To plug your own index into an assistant, expose it through an MCP server (later).
Context window vs. 8,192?	8,192 (8,191) is the max input tokens for OpenAI embedding models. The context window is the max input for a chat model. Both differ from the embedding dimension.
Which search did yesterday's RAG use?	`search_type="similarity"`: pure vector (cosine) search over HNSW, not hybrid.

Question	Answer
Chatbot with RAG + LLM API – what runs first?	Retrieval runs first (the query embedding uses the embedding API), then the chat LLM API generates from the retrieved context. The API is just how you call each model.
What does the pipe `|` in the chain do?	Connects steps: retriever (R) → prompt (A) → LLM (G). Each output feeds the next.
Is HNSW followed by a cross-encoder?	HNSW makes vector search fast across millions of vectors. A cross-encoder re-ranker can then reorder the top results, but only if you add it.
Gemini embedding errors on the free tier?	Free tiers have rate and quota limits; batch your requests, add retries, or use a paid tier.
Is the user's query embedded too, and is it billed?	Yes. Every query is embedded (billed per token), then searched. It isn't stored in the vector DB.
VS Code or Colab?	Colab for learning (free GPU if needed). For heavier local models, rent a cloud GPU (e.g. RunPod) and shut it down after use.
Will interviews ask deep ML/DL?	Rarely for GenAI roles. Don't claim deep expertise you can't defend; steer to RAG/agents.

8. Expert corrections to the lecture

1. **BM25 *is* keyword search.** It's the standard keyword-ranking algorithm (term frequency, inverse document frequency, length normalisation) and uses no embeddings. It is the keyword half of hybrid search, not a method to avoid.
2. **Vector search and semantic search are the same thing.** Hybrid = keyword (BM25) + vector/semantic, not three separate searches.
3. **Hybrid search isn't for "text + images/pixels".** It's about combining lexical and semantic matching over text. Images need multimodal embeddings.
4. **Yesterday's code was not hybrid.** `search_type="similarity"` is pure vector search. Hybrid needs e.g. LangChain's `EnsembleRetriever` (BM25 + vector) or a DB with built-in hybrid search.

5. **RRF merges ranked lists using rank positions,** without any LLM. RAG-Fusion is the separate technique that adds LLM-generated query variants. Re-ranking uses relevance scores, not probabilities.
6. **8,192 is an input token limit, not an embedding dimension.** text-embedding-3-small has 1,536 dimensions, text-embedding-3-large 3,072. The original Transformer used 512 (the lecture said 524).
7. **Parameters ≠ features ≠ dimensions.** The input limit doesn't determine parameter count. Corpus sizes are measured in tokens, not "trillions of documents".
8. **Similarity isn't computed from an "average value".** Euclidean, cosine and dot product are computed across all dimensions of the two vectors.
9. **Tokenisation splits text into tokens/IDs;** the neural network then produces the embedding. Tokenisation alone isn't "converting to numbers" in the embedding sense.
10. **BoW stores counts (not only 0/1); TF-IDF values are weights, not probabilities.**
11. **Dates and labs:** GloVe was 2014. The Transformer came from Google Brain/Google Research (not DeepMind). GPT-3 had 175B parameters.
12. **Backpropagation** was popularised in 1986 (Rumelhart, Hinton, Williams). Hinton's 2006 deep-belief-network paper revived deep learning.
13. **Fine-tuning ≠ RLHF.** Fine-tuning means further training a pre-trained model on task data; RLHF is one particular fine-tuning method using human preferences.
14. **Uploading files to ChatGPT/Copilot often does use retrieval internally,** so it's a form of (managed) RAG, not only a plain LLM call.

9. Interview quick revision and career tips

30-second answers

> **What is an embedding?** A dense vector produced by a neural model that captures meaning, so similar texts sit close together. We compare vectors with cosine similarity. Unlike one-hot/BoW/TF-IDF, embeddings capture synonyms and context.

> **What is hybrid search?** Running BM25 keyword search (exact terms, IDs) and vector search (meaning) in parallel, then merging the lists, usually with Reciprocal Rank Fusion, often followed by a cross-encoder re-ranker. It's the default for enterprise RAG.

> **Your RAG gives poor answers – what do you do?** Check chunking, move to hybrid search, add re-ranking, try query rewriting/RAG-Fusion, evaluate each change, and only then consider fine-tuning.

Practice suggested in class

- [ ] Ask a chatbot for "top 20 RAG interview questions" and answer them aloud; most are now covered
- [ ] Write the RAG flow and the search-method table from memory in your own words
- [ ] Rewatch Prompting, LangChain and Simple RAG if you joined late

Resume advice from the lecturer: four projects

1. A **cloud-based** RAG/agent project (Azure, AWS or GCP)
2. A **RAG-from-scratch** project (multimodal, graph or agentic RAG)
3. An **MCP** project, to show recent skills
4. A project from **your own domain** that connects to LLM/RAG

Use bullet points, not stories, and only claim what you can defend in an interview.

SESSION 5

Saturday, 20 June 2026

RAG App, Latency, Multimodal RAG & Agents Intro

RAG Notes – Session 5 (20 June 2026)

Oct 14, 2026 · @Vljay

1. Building a RAG chatbot app (Streamlit + FAISS)

This turns the notebook RAG into a **POC web app**: upload a PDF, ask questions, get answers grounded only in that PDF. In real projects a POC like this is shown to the client before the full build is approved.

Layer	Choice in class	Alternatives
Front end	Streamlit	Flask, React
PDF reading	`PdfReader` (pypdf/PyPDF2)	PyMuPDF, LangChain loaders
Chunking	`RecursiveCharacterTextSplitter` (1,000 / 100 overlap)	Semantic, LLM-based
Embeddings	`OpenAIEmbeddings`	Gemini, Hugging Face
Vector store	FAISS (Facebook AI Similarity Search), saved locally – good for POCs	Chroma, Pinecone, Azure AI Search
LLM	`ChatOpenAI(model="gpt-4o-mini", temperature=0.3)`	Gemini 2.5 Flash, Groq

Pipeline as functions (app.py)

```
import streamlit as st
from pypdf import PdfReader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate

def get_pdf_text(pdf_docs):
    text = ""
    for pdf in pdf_docs:
        for page in PdfReader(pdf).pages:
            text += page.extract_text() or ""
    return text
```

```
def get_text_chunks(text):
    splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=100)
    return splitter.split_text(text)

def build_vector_store(chunks, api_key):
    emb = OpenAIEmbeddings(api_key=api_key)
    FAISS.from_texts(chunks, embedding=emb).save_local("faiss_index")

PROMPT = ChatPromptTemplate.from_template(
    "Answer the question as detailed as possible from the provided context. "
    "If the answer is not in the context, say \"Answer is not available in
the context\"; "
    "do not provide a wrong
answer.\n\nContext:\n{context}\n\nQuestion:\n{question}\n\nAnswer:"
)

def answer(question, api_key):
    emb = OpenAIEmbeddings(api_key=api_key)
    db = FAISS.load_local("faiss_index", emb,
allow_dangerous_deserialization=True)
    docs = db.similarity_search(question, k=4)
    context = "\n\n".join(d.page_content for d in docs)
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.3, api_key=api_key)
    return (PROMPT | llm).invoke({"context": context, "question":
question}).content

def main():
    st.set_page_config(page_title="RAG Chatbot")
    st.header("RAG Chatbot with Streamlit and OpenAI")
    with st.sidebar:
        st.title("Menu")
        api_key = st.text_input("Enter your OpenAI API key", type="password")
        pdfs = st.file_uploader("Upload PDF files",
accept_multiple_files=True)
        if st.button("Submit & Process") and api_key and pdfs:
            with st.spinner("Processing..."):
                build_vector_store(get_text_chunks(get_pdf_text(pdfs)),
api_key)
                st.success("Done")
    question = st.text_input("Ask a question from the PDF files")
    if question and api_key:
        st.write("Reply:", answer(question, api_key))

if __name__ == "__main__":
    main()
```

Run it with `streamlit run app.py`.

Demo results (Attention paper, 11 pages)

- "Explain the transformer paper in five simple lines for non-tech people" → clear, grounded summary.
- "What is self-attention?" → correct, even with spelling mistakes in the question.
- "Who is Narendra Modi?" / "What is Newton's law?" → "Answer is not available in the context" (no hallucination).

Homework extension: add loaders for Excel, CSV and images; try another chunking strategy; swap FAISS for Chroma (persists to disk); store Q&A history; show an evaluation score per answer (target 90–100%).

2. Reducing latency in a RAG app

Latency = how long the user waits for an answer. Each question triggers: embed the query → search the vector DB → build the prompt → LLM generation. The lecture said this question is asked in nearly every interview: "Your bot takes minutes to answer – what do you do?"

#	Lever	What to do
1	Chunking	Revisit the strategy and size; avoid huge numbers of tiny chunks, and avoid oversized chunks that bloat the prompt
2	Caching	Store answers to repeated/FAQ questions in a cache (e.g. **Redis**) and skip retrieval + generation on a hit; a semantic cache also matches paraphrased questions
3	Retrieval efficiency	Use an ANN index (HNSW), a sensible k, metadata filters; hybrid search + re-ranking only on a small candidate set
4	Embedding dimensions	Use smaller dimensions (e.g. 256–512) for faster search and less storage
5	Model choice	A smaller/faster LLM (e.g. a mini model), shorter prompts, streaming output so users see tokens immediately
6	Infrastructure	Keep the index in memory, scale the vector DB, run async or batch calls

Memory vs. cache: a cache returns stored answers to repeat questions; conversation memory stores chat history so follow-ups ("where does *he* live?") make sense. Both come up again in agentic AI.

3. Multimodal RAG

Core idea: convert every element of a document into text an LLM can search and reason over, then run the normal RAG pipeline on it. The class example was a 3-page wildfire report with text paragraphs, charts (line/area graph of fires and acres burned) and tables.

Element	How to handle it	Result
Text	Extract directly (PyMuPDF, pypdf, Unstructured)	Text chunks
Images / charts	Extract the image, then describe it with a **vision-language model** (GPT-4o, LLaVA). OCR only pulls any printed text out of the image	**Image summary** in words (e.g. "line chart: fires in thousands, 1993–2022; area: acres burned in millions")

Element	How to handle it	Result
Tables	Extract as **HTML → convert to Markdown** (e.g. `html2text` / `markdownify`) → optionally summarise with an LLM	**Table summary** + Markdown table

Then: chunk → embed (summaries) → vector DB → retrieve → augment → generate. Keep a link from each summary back to its original image/table so answers can cite or show it.

Interview answer: "For PDFs with text, images and tables, I'd parse them by element. Text goes straight in. Images are summarised with a multimodal model, with OCR for any embedded text. Tables become HTML, then Markdown, then a summary. I embed the summaries, keep the originals in a doc store, add caching (Redis) for latency, and run normal RAG with citations."

Multimodal models and tools

Tool	What it is
GPT-4o / GPT-4o mini	OpenAI "omni" model: text, image, audio input
Gemini 2.5 Pro	Google multimodal model
LLaVA	Open-source vision-language model ("Large Language and Vision Assistant"); ~20 GB GPU to run locally
Fuyu-8B	Adept's open multimodal model for agents
Unstructured.io	Parses 60+ file types into elements (text, table, image), with chunking by title and metadata; free trial, then pay-as-you-go per page
Tesseract + Poppler	OCR engine + PDF rendering utilities used by Unstructured
Hugging Face	Hub hosting ~2.9M models; read model cards and GitHub READMEs

Setup (Colab, GPU runtime)

```
!pip install langchain langchain-openai langchain-community langchain-chroma
redis
!pip install "unstructured[pdf]" pdfminer.six pillow-heif htmltabletomd   #
or unstructured[all-docs]
!sudo apt-get install -y tesseract-ocr poppler-utils
```

Loading a mixed PDF with Unstructured

```
from langchain_community.document_loaders import UnstructuredPDFLoader

loader = UnstructuredPDFLoader(
    file_path="wildfire_report.pdf",
    strategy="hi_res",                    # layout model + OCR: best quality,
slower
    extract_images_in_pdf=True,           # save images
    infer_table_structure=True,           # keep tables as HTML
    chunking_strategy="by_title",         # section-based chunks
    max_characters=1000,                  # hard cap per chunk
    new_after_n_chars=1000,               # soft cap
    combine_text_under_n_chars=500,       # merge tiny sections
    mode="elements",                      # one Document per element
    image_output_dir_path="./figures",
)
data = loader.load()
```

Lessons from the live debugging

The demo hit dependency errors (Pillow, pdfminer.six, Unstructured versions), so the multimodal pipeline was postponed to the next session.

- Pin compatible versions together (`unstructured[pdf]`, `pdfminer.six`, `pillow`), install in separate cells, restart the runtime after big installs.
- Files uploaded to Colab can be lost or zeroed after a runtime reset; re-upload, or mount Google Drive.
- In companies, packages often need IT whitelisting; managed cloud services avoid much of this.
- Indexing (parsing messy documents) is usually the hardest part of real RAG.

4. Agentic AI: the four components

Agentic AI is a system with the agency to pursue a goal on its own: it plans, uses tools, remembers, and takes actions, not just answers. Progression: LLM (public knowledge) → RAG (your data) → agents (actions + tools).

Component	Role	Analogy	Example
Model (LLM)	Understands language, decides what to do next, writes the answer	Brain	Reads "cheapest flight Bangalore → Delhi" and extracts origin, destination, constraint
Tools	Connect to the outside world: APIs, web search, databases, RAG retrievers, MCP servers	Hands	Searches a travel site for flights, fares, classes
Memory	Keeps context across turns and sessions	Notebook	Remembers "my name is Sundaram" or last trip's price
Planning & reasoning	Breaks a goal into steps, chooses actions, checks progress	Manager	Step 1 find flights → step 2 compare fares → step 3 ask human before paying

Without a model there's no intelligence; without tools no action or live data; without memory every chat starts from zero; without planning it can only react one step at a time.

How they interact: the LLM never calls a tool or reads memory directly. The **agent runtime** does: the LLM proposes a tool call, the agent executes it and feeds the result back.

Memory types (detail later)

- **Short-term:** current conversation / working context (like a cache).
- **Long-term:** persisted across sessions (e.g. in a vector DB): semantic (facts), episodic (past interactions), procedural (how to do things).

Example: summarise a PDF

1. The user asks for a summary of a PDF.
2. Planning (a chain-of-thought style plan): read the PDF → extract text → highlight key points → summarise.
3. Tool: fetch the PDF from the company repository (e.g. via an MCP server).

4. Model: writes the summary.
5. Memory: stores the summary for later ("what did the last 6 months of project emails say?").

MCP in one line

Model Context Protocol = a standard way to expose tools and data (often live) to LLM apps. Model = LLM, context = information, protocol = set of rules. It solves "the LLM's training data stops at a cutoff, but I need today's data".

Agentic RAG

The retriever over your vector DB becomes **one of the agent's tools**. The agent decides when to retrieve, can answer FAQs from memory/cache instead, and can combine retrieval with other tools.

5. Agent patterns

Complex tasks are split across several agents. The five patterns from class (hands-on next session):

Pattern	How it works	Example from class
Single agent	One agent handles everything	One HR assistant over all company policy documents
Two-agent (collaboration)	Two agents converse until satisfied: e.g. a doer and a reviewer	A coder agent writes code; a senior-reviewer agent checks it before deployment. Or a planner agent plus a booking agent, with a human approving payment
Sequential	Each agent finishes before the next starts; output flows down the line	Agent 1 load/chunk/embed → Agent 2 RAG answer → Agent 3 evaluation → Agent 4 guardrails/observability
Group (orchestrator / router)	A manager agent routes each request to a specialist	Orchestrator sends leave questions to HR, loans to Finance, updates to Comms, campaigns to Marketing
Nested	An agent runs its own inner agents or checks before answering	The HR agent checks whether a question asks for confidential data (someone's salary, phone, address) and refuses

Human-in-the-loop: for irreversible actions like payments or deployments, require human approval in the plan.

6. Q&A and career highlights

Question	Answer
A job asks for GCP but I know Azure.	Concepts transfer. Say you've worked on one cloud and the services map across; tools are learned on the job.
Architect role vs. coding focus?	Experienced people should aim for solution/principal architect: design, security, logging, observability, memory, tool calling, MCP. Coding is a reference; Copilot and Cursor write much of it.
Snowflake/Databricks/AI Foundry?	Focus on concepts; specific platforms are quick to pick up. Azure (Foundry, Cosmos DB) is covered later in the course.
Word- vs sentence-level embedding?	Embed whole chunks (here ~1,000 characters), not individual words.
Can I store conversation history?	Yes. Persist Q&A (e.g. Chroma on disk) for memory and later evaluation.
Can I use Gemini instead of OpenAI?	Yes. Use Gemini 2.5 Flash (not 1.5) and adjust syntax; Groq is another free option. LearnBay may provide API keys.
Generic loader vs. one loader per file type?	Either works. Per-type loaders give more control over metadata and parsing quality.
How do I chunk an Excel file with numbers + text?	Convert rows/tables to text (e.g. Markdown) first, then chunk.
What is CAG / C-RAG?	Corrective RAG and other variants come later (see correction 9).
What is evaluation going to cover?	Context precision, recall and relevancy; LLM-as-judge; hallucination, faithfulness, answer relevancy (DeepEval/RAGAS); a golden dataset from the client.
Resume projects?	(From Session 4) cloud project, RAG project, MCP project, domain project.

Question	Answer
Schedule	Saturday 10:30–13:30, Sunday 10:00–13:30. Weekly assignments; optional 10–15 min 1:1 slots.

7. Expert corrections to the lecture

1. **OCR reads printed characters; it does not describe pictures.** Turning a chart or a photo of a dog into a sentence ("a black dog on green grass") needs a vision-language model (GPT-4o, LLaVA, Gemini). OCR (Tesseract) only extracts text that appears inside the image.
2. **Redis is a cache / key-value store, not part of OCR.** In multimodal RAG it's typically used as a fast doc store for originals or as a cache.
3. **"Chunk size 5,000 because the model reads 8,191 tokens" mixes up limits.** 8,191 tokens is the *embedding model's* input limit. Bigger chunks reduce retrieval precision and inflate prompts; they aren't a latency fix.
4. **Order of operations:** retrieval (hybrid: BM25 + vector) produces candidates first, then re-ranking reorders them. Not "rank first, then hybrid".
5. **Similarity search compares full vectors** (cosine/distance), not "average numbers".
6. **`load_qa_chain(chain_type="stuff")`** (not "a stop") is legacy. Prefer LCEL (`prompt | llm`) or `create_stuff_documents_chain`.
7. **`allow_dangerous_deserialization=True`** loads pickled files. Only use it on FAISS indexes you created yourself.
8. **GPT-4o's "o" means omni;** Gemini's fast model is 2.5 **Flash** (not "Flask"); FAISS = Facebook AI Similarity Search.
9. **CAG usually means Cache-Augmented Generation:** preloading a small, stable knowledge base into the model's context/KV cache instead of retrieving per query. Corrective RAG (CRAG) is a different technique that grades retrieved docs and re-searches if they're poor.
10. **Planning isn't only chain-of-thought,** and reasoning isn't "permutation and combination". Agents plan via task decomposition (ReAct, plan-and-execute) and reason by evaluating options and tool results.
11. **The "nested" agent pattern** (e.g. AutoGen nested chats) means an agent runs inner conversations. Guarding confidential data must be enforced in the tools/data layer (access control), not just with an if/else in a prompt.

12. **Unstructured parameter names:** `extract_images_in_pdf` / `extract_image_block_types`, `infer_table_structure`, `chunking_strategy="by_title"`, `max_characters`, `new_after_n_chars`, `combine_text_under_n_chars`, `mode="elements"`. The table-to-Markdown package is `htmltabletomd` (or use `markdownify`).
13. **Embeddings + vector DB form a separate step from tokenisation.** Tokens are produced inside the embedding model; you don't tokenise manually before embedding.

8. Interview quick revision and homework

30-second answers

> **What is agentic AI?** A system that pursues a goal autonomously. An LLM "brain" plans and reasons, calls tools (APIs, search, RAG retrievers, MCP servers) through the agent runtime, and uses short- and long-term memory, taking actions rather than only answering. Human approval gates risky steps.

> **How would you build RAG over PDFs with images and tables?** Parse by element (e.g. Unstructured, hi_res). Text as-is. Images summarised by a multimodal LLM, plus OCR for embedded text. Tables via HTML → Markdown → summary. Embed the summaries, keep the originals linked, then run standard RAG with citations and caching.

> **Your RAG is slow. Fix it.** Tune chunking, cache repeat questions (Redis/semantic cache), use an HNSW index with sensible k and filters, reduce embedding dimensions, use a faster model with streaming, and re-rank only a small candidate set.

Homework

- [ ] Run the Streamlit RAG app; extend it to Excel/CSV/images and swap FAISS for Chroma
- [ ] Research **AI agent vs. agentic AI** for the next class (hint: an AI agent is a single goal-directed component; agentic AI is the broader capability/system of autonomous, often multi-agent, behaviour)
- [ ] Complete the posted assignments (prompt engineering, LangChain, RAG) without AI coding assistants
- [ ] Write your own story for the latency question

Coming next

Finish the multimodal RAG code (after dependency fixes), Redis caching, then hands-on agent patterns (AutoGen, CrewAI, LangGraph), MCP, Graph RAG, fine-tuning (Transformer → BERT → PEFT → LoRA/QLoRA).

SESSION 6

Sunday, 21 June 2026

Multimodal RAG Build & Redis

RAG Notes – Session 6 (21 June 2026)

Oct 14, 2026 · @Vljay

1. AI agent vs. agentic AI

Interviewers often ask for the difference. Use the version in the right-hand column.

	Lecture's framing	More precise industry framing
AI agent	Responds when asked: question → LLM → tool/API → answer. Examples given: ChatGPT, Gemini, Claude, Perplexity	A single goal-directed component: an LLM that can use tools to complete a specific task
Agentic AI	A system you build where an agent coordinates **model, memory, tools, planning & reasoning**	The broader paradigm or system: one or many agents acting autonomously over multiple steps, with planning, memory, orchestration and (often) human-in-the-loop

One-liner: An AI agent is the worker; agentic AI is the autonomous system or way of working built from such agents.

The four pillars don't talk to each other directly. The agent (runtime) coordinates them. Platforms such as Rovo (Atlassian), with connectors to Jira and Confluence, count as agentic platforms.

2. Multimodal RAG pipeline end to end

Vanilla (traditional) RAG has one input type. Multimodal RAG has several: text, images, tables, and also audio or video. Class data: a 3-page wildfire report with 14 text chunks, 2 tables and 5 images.

1. **Partition the PDF** with Unstructured (`hi_res`): text, tables (as HTML) and images (saved to `./figures`), chunked **by title** → 14 text chunks (`CompositeElement`) + 2 `Table` elements.
2. **Tables:** HTML → Markdown (`htmltabletomd`), because LLMs read Markdown tables reliably.
3. **Summarise text and tables** with GPT-4o (one prompt, batched).
4. **Summarise images** with GPT-4o vision (images sent as base64).

5. **Index with a multi-vector retriever:** embed the *summaries* into Chroma (vector store); store the *original* text, tables and images in Redis (doc store), linked by a shared `doc_id`.
6. **Retrieve:** query → similar summaries in Chroma → fetch the matching originals from Redis.
7. **Split** the retrieved items into text/tables vs. images.
8. **Augment + generate:** build a multimodal prompt (text + tables + images) for GPT-4o → answer.
9. **Return sources:** the answer plus the text, tables and images it used (citations).

The idea behind step 5: search on clean, retrieval-friendly summaries, but answer from the full original content.

3. Code part 1: partition and summarise

Dependency fix from Session 5

```
!pip install "unstructured[pdf]" pdfminer.six==20221105 htmltabletomd
!sudo apt-get install -y tesseract-ocr poppler-utils
# restart the runtime after installing
```

Partition the PDF

```
from langchain_community.document_loaders import UnstructuredPDFLoader

loader = UnstructuredPDFLoader(
    file_path="wildfire_report.pdf",
    strategy="hi_res",
    extract_images_in_pdf=True,
    infer_table_structure=True,
    chunking_strategy="by_title",
    max_characters=4000,            # class used 1,000–4,000; keep well under
~8K tokens
    new_after_n_chars=4000,
    combine_text_under_n_chars=2000,
    mode="elements",
    image_output_dir_path="./figures",
)
data = loader.load()                # 16 Documents: 14 text + 2 tables

docs, tables = [], []
for doc in data:
    if doc.metadata["category"] == "Table":
```

```
        tables.append(doc)
    elif doc.metadata["category"] == "CompositeElement":
        docs.append(doc)
len(docs), len(tables)              # (14, 2)
```

Every element is a LangChain `Document` (`page_content` + `metadata`). For tables, `metadata["text_as_html"]` holds the structured table. Filtering by `metadata["category"]` scales to thousands of pages.

Tables → Markdown

```
import htmltabletomd
for table in tables:
    table.page_content =
htmltabletomd.convert_table(table.metadata["text_as_html"])
```

Skipping this gives poor answers on table questions (the lecture estimated ~70% accuracy instead of 90%+).

Summarise text and tables

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_template("""
You are an assistant tasked with summarizing tables and text particularly for
semantic retrieval.
These summaries will be embedded and used to retrieve the raw text or table
elements.
Give a detailed summary of the table or text below that is well optimized for
retrieval.
For any tables also add a one-line description of what the table is about
besides the summary.
Do not add redundant words like "Summary". Just output the actual summary
content.

Table or text chunk:
{element}
""")

chatgpt = ChatOpenAI(model="gpt-4o", temperature=0)
summarize_chain = {"element": lambda x: x} | prompt | chatgpt |
StrOutputParser()
```

```
text_docs = [d.page_content for d in docs]
table_docs = [t.page_content for t in tables]
text_summaries = summarize_chain.batch(text_docs, {"max_concurrency": 5})
table_summaries = summarize_chain.batch(table_docs, {"max_concurrency": 5})
```

`max_concurrency=5` runs up to 5 summaries **in parallel** to save time.

Result (table 1): the model correctly described fires and acres burned, 2018–2022, split by federal (FS, DOI, other) vs non-federal, including peaks (e.g. total fires peaking around 69K).

Summarise images with GPT-4o vision

```
import base64, os
from langchain_core.messages import HumanMessage

def encode_image(path):
    with open(path, "rb") as f:
        return base64.b64encode(f.read()).decode("utf-8")

def image_summarize(img_b64, prompt):
    msg = chatgpt.invoke([HumanMessage(content=[
        {"type": "text", "text": prompt},
        {"type": "image_url", "image_url": {"url": f"
{img_b64}"}},
    ])])
    return msg.content

IMG_PROMPT = ("You are an assistant tasked with summarizing images for
retrieval. "
              "These images could contain graphs, charts or tables. This
summary will be embedded "
              "and used to retrieve the raw image for question answering.
Give a detailed summary "
              "of the image that is well optimized for retrieval. Do not add
redundant words like Summary.")

img_base64_list, image_summaries = [], []
for fname in sorted(os.listdir("figures")):
    if fname.lower().endswith((".jpg", ".jpeg", ".png")):
        b64 = encode_image(os.path.join("figures", fname))
        img_base64_list.append(b64)
        image_summaries.append(image_summarize(b64, IMG_PROMPT))
```

Result (image 1): "Line and area chart of fires and acres burned, 1993–2022; left y-axis = fire occurrences, right y-axis = acres burned (millions); red line peaks around 1996 and 2006; grey area peaks around 2006, 2012 and 2020." That is a human-quality reading of the chart.

4. Code part 2: multi-vector retriever and RAG chain

Set up Redis + Chroma

```
!sudo apt-get install -y redis-stack-server   # or run Redis in Docker / a
managed service
!pip install redis langchain-chroma

import uuid
from langchain.retrievers.multi_vector import MultiVectorRetriever
from langchain_community.storage import RedisStore
from langchain_community.utilities.redis import get_client
from langchain_chroma import Chroma
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

vectorstore = Chroma(
    collection_name="mm_rag",
    embedding_function=embeddings,
    collection_metadata={"hnsw:space": "cosine"},
)
redis_store = RedisStore(client=get_client("redis://localhost:6379"))
```

Build the multi-vector retriever

```
def create_multi_vector_retriever(docstore, vectorstore,
                                  text_summaries, texts,
                                  table_summaries, tables,
                                  image_summaries, images):
    id_key = "doc_id"
    retriever = MultiVectorRetriever(vectorstore=vectorstore,
docstore=docstore, id_key=id_key)

    def add_documents(summaries, contents):
        doc_ids = [str(uuid.uuid4()) for _ in contents]
        summary_docs = [Document(page_content=s, metadata={id_key:
doc_ids[i]})
```

```
                        for i, s in enumerate(summaries)]
        retriever.vectorstore.add_documents(summary_docs)          #
summaries -> Chroma
        retriever.docstore.mset(list(zip(doc_ids, contents)))      #
originals -> Redis

    if text_summaries:  add_documents(text_summaries, texts)
    if table_summaries: add_documents(table_summaries, tables)
    if image_summaries: add_documents(image_summaries, images)
    return retriever

retriever_multi_vector = create_multi_vector_retriever(
    redis_store, vectorstore,
    text_summaries, text_docs,
    table_summaries, table_docs,
    image_summaries, img_base64_list,
)

docs = retriever_multi_vector.invoke("Analyze the wildfires trend with acres
burned over the years")
```

Split retrieved items into images vs. text

```
import base64, re

def looks_like_base64(sb):
    return re.match(r"^[A-Za-z0-9+/]+[=]{0,2}$", sb) is not None

def is_image_data(b64data):
    sigs = {b"\xff\xd8\xff": "jpg", b"\x89\x50\x4e\x47": "png",
b"\x47\x49\x46\x38": "gif", b"\x52\x49\x46\x46": "webp"}
    try:
        header = base64.b64decode(b64data)[:8]
        return any(header.startswith(s) for s in sigs)
    except Exception:
        return False

def split_image_text_types(docs):
    images, texts = [], []
    for doc in docs:
        doc = doc.decode() if isinstance(doc, bytes) else doc   # Redis
returns bytes
        if looks_like_base64(doc) and is_image_data(doc):
```

```
            images.append(doc)
        else:
            texts.append(doc)
    return {"images": images, "texts": texts}
```

Multimodal prompt + chain with sources

```
from operator import itemgetter
from langchain_core.runnables import RunnableLambda, RunnablePassthrough

def multimodal_prompt_function(data_dict):
    formatted_texts = "\n".join(data_dict["context"]["texts"])
    messages = [{"type": "image_url", "image_url": {"url":
f"{img}"}}
                for img in data_dict["context"]["images"]]
    messages.append({"type": "text", "text": f"""You are an analyst tasked
with understanding detailed
information and trends from text documents, data tables, and charts and
graphs in images.
You will be given context information below which will be a mix of text,
tables, and images.
Use this information to answer the user question. Analyze all the context
information including
tables, text and images to generate the answer. Do not make up answers; use
only the provided
context and answer the question to the best of your ability.

User question:
{data_dict['question']}

Context documents:
{formatted_texts}

Answer:"""})
    return [HumanMessage(content=messages)]

multimodal_rag = (
    {"context": itemgetter("context"), "question": itemgetter("input")}
    | RunnableLambda(multimodal_prompt_function)
    | chatgpt
    | StrOutputParser()
)

retrieve_docs = (itemgetter("input") | retriever_multi_vector |
RunnableLambda(split_image_text_types))
```

```
multimodal_rag_w_sources = (
    {"input": RunnablePassthrough()}
    | RunnablePassthrough.assign(context=retrieve_docs)
    | RunnablePassthrough.assign(answer=multimodal_rag)
)

response = multimodal_rag_w_sources.invoke({"input": "Tell me detailed
statistics of the top 5 years with largest wildfire acres burned"})
print(response["answer"])          # answer
response["context"]["texts"]       # text/table sources
response["context"]["images"]      # image sources (display with IPython)
```

Demo results: correct top-5 years (2015, 2020, 2017, 2006, 2007) with acres burned and fire counts, citing the bar-chart image. Correct 2020 figures from table 1. "Who won IPL 2026?" → "I can't provide that based on the given context."

5. Redis: doc store vs. cache

Redis is a high-speed, in-memory key-value (NoSQL) database. Typical uses: caching, session management, real-time analytics, queues, and vector storage (with the Redis Stack search module). It supports strings, hashes, lists, sets and more.

In this notebook Redis is the doc store, not an answer cache. Two roles get mixed up easily:

Role	What's stored	When it's used	Set up in class?
Doc store (MultiVectorRetriever)	Original text chunks, Markdown tables, base64 images, keyed by `doc_id`	On *every* query: Chroma finds matching summaries, then their originals are fetched from Redis	**Yes**
LLM / semantic cache	Prompt → answer pairs	Before calling the LLM: a hit returns the stored answer immediately	No; must be added explicitly

Vector store vs. doc store: the vector store (Chroma) holds embeddings for similarity search; the doc store (Redis) holds the full original content looked up by ID. They really are different components.

Adding a real answer cache

```
from langchain.globals import set_llm_cache
from langchain_community.cache import RedisCache, RedisSemanticCache
import redis

set_llm_cache(RedisCache(redis_=redis.Redis()))          # exact-match
prompts
# or
set_llm_cache(RedisSemanticCache(redis_url="redis://localhost:6379",
                                 embedding=embeddings,
                                 score_threshold=0.1))   # similar prompts
```

Benefits of caching (interview answer)

1. **Lower cost:** fewer LLM and embedding calls.
2. **Faster responses:** answers come from memory, no retrieval or generation.
3. **Fewer LLM calls:** if 100 users ask the same FAQ, the LLM answers once.
4. **Scales** for high traffic.

Watch out: cached answers can go stale when documents change. Use TTLs (expiry times) and clear the cache when you re-index.

In Azure, the equivalents are Azure Cache for Redis and Cosmos DB (both covered later). Cloud storage is billed, whereas Colab memory disappears when the session ends.

6. Q&A and interview questions

Question	Answer
Chroma vs. Redis?	Chroma = vector DB (embeddings of summaries, similarity search). Redis = key-value doc store holding the originals.
Is the output always text?	It's whatever the chain returns. Here `StrOutputParser` makes it a string; you can return structured JSON instead.
Is `max_concurrency=5` like epochs?	No. It just runs up to 5 requests in parallel. Nothing is trained.

Question	Answer
If I correct a fact, will an old cached answer be returned?	With a real answer cache, possibly. Answers come from your indexed documents, so update and re-index the documents and clear/expire the cache.
How does the system know whether to use Redis or Chroma?	In this design it always searches Chroma, then fetches originals from Redis by ID. A cache layer (if added) is checked first in code, not decided by the LLM.
How does the retriever handle paraphrased questions?	The query is embedded and matched by meaning against summary embeddings.
Chroma or Pinecone in real projects?	Both are used, plus Azure AI Search and others; it depends on the client's stack.
If 100 users ask the same question?	Add an answer cache (Redis/semantic cache) so the LLM runs once.
If I add an unrelated document (e.g. IPL results)?	After indexing, the bot will answer from it. The knowledge base defines the scope, so curate it.
Where do loaded documents live before the vector DB?	In Python objects in memory (`docs` , `tables` , files in `./figures`) until you index them.
Interview: Why does LangChain exist if providers have APIs?	It's a framework/abstraction layer: a uniform interface over many LLMs, loaders, splitters, vector stores and retrievers, plus chaining (LCEL). You can swap Gemini for OpenAI without rewriting the app.
Interview: 100 PDFs – how do you ensure relevant answers?	Careful preprocessing (clean parsing, good chunking, rich metadata such as a document catalog of title/topic/date for filtering), hybrid search + re-ranking, an "answer only from context" prompt, evaluation against a client golden dataset.
Agent conversation vs. A2A protocol?	Agents chatting (two-agent, group…) is a design pattern. A2A (Agent-to-Agent) is a protocol standard for agents from different systems to communicate, like MCP is for tools. Covered later.

Question	Answer
Do I need to memorise code?	No. Know the architecture and concepts; know Python for written tests; real work is mostly on cloud platforms.

7. Expert corrections to the lecture

1. **Redis here is a doc store, not an answer cache** (the biggest correction). The code embeds summaries into Chroma and stores originals in Redis. Every query still searches Chroma first. Generated answers are **not** written to Redis, and Redis isn't checked "first". Caching answers requires `set_llm_cache(RedisCache/RedisSemanticCache)` or custom code.
2. **The "IPL response came from Redis" demo was not memory.** The chain has no conversation history. The model simply answered the new question from the retrieved context.
3. **Redis stores raw documents (bytes/strings), not embeddings,** in this setup. Embeddings live only in Chroma.
4. **Doc store vs. vector store differ:** the doc store looks up content by ID; the vector store searches by similarity.
5. **`max_concurrency=5` = parallelism.** It is not "iterating five times", refining the output, or fine-tuning.
6. **Images are described by GPT-4o's vision capability,** not OCR. Base64 is just how image bytes are sent over the API. The LLM doesn't "read pixel values" the way a classic CNN pipeline does.
7. **AI agent vs. agentic AI:** calling ChatGPT/Gemini "just AI agents, not agentic" is outdated. These products now include agent modes with tools and memory. The cleaner distinction is component (agent) vs. autonomous system/paradigm (agentic AI).
8. **The retriever here is pure vector search,** not hybrid. The prompt's phrase "semantic retrieval" refers to embedding search.
9. **Colours:** 8-bit RGB gives $256^3 \approx 16.7$ million colours.
10. **Limits:** OpenAI embedding models accept ~8K tokens; text-embedding-3-large defaults to 3,072 dimensions (not "3,000"). Very large chunks (5–8K) usually hurt retrieval precision.
11. **Azure's main vector option is Azure AI Search** (or Cosmos DB vector search), not Chroma.

12. **LSTM memory isn't a fixed "30 words".** LSTMs struggle with long-range dependencies; Transformers solved this with attention.

8. Quick revision and assignments

60-second answer: "Walk me through multimodal RAG"

> I partition PDFs with Unstructured (hi_res) into text, tables and images, chunking text by title. Tables go from HTML to Markdown. I use GPT-4o to write retrieval-optimised summaries of text chunks, tables and images (images sent as base64). A MultiVectorRetriever embeds the summaries into a vector store (Chroma, cosine HNSW) and keeps the originals in a doc store (Redis), linked by doc_id. At query time I retrieve summaries, fetch the originals, split images from text, and send both to GPT-4o with an "answer only from context" prompt. The answer comes back with its source text, tables and images. For latency I'd add a Redis semantic cache with TTLs.

Assignments (folder "Assignment for learning")

- [ ] **Assignment 1 – prompt-only pricing agent:** build an LLM price recommender using prompt techniques only (no RAG/tools)
- [ ] **Assignment 2 – RAG over three papers:** "Attention Is All You Need", the RAG paper, and "Language Models are Few-Shot Learners"; answer the five given questions. Try the multimodal version, since the papers contain figures
- [ ] Re-run the multimodal notebook end to end and inspect `response["context"]`
- [ ] Add a real Redis semantic cache and measure the speed-up on repeated questions

Coming next

Agentic AI hands-on (single, two-agent, sequential, group, nested), agentic RAG, then fine-tuning, evaluation metrics, Graph RAG, CRAG, cloud (Azure), MCP and A2A, deep learning basics.

Schedule: Saturday 10:30, Sunday 10:00. Book 1:1s through the LearnBay coordinator; don't create side groups.

SESSION 7

Saturday, 27 June 2026

Agent Frameworks: LangGraph, CrewAI, AutoGen

Agentic AI Notes – Session 7 (27 June 2026)

Oct 14, 2026 · @Vljay

1. Why multi-agent systems

One employee can't do a whole project, and one LLM call shouldn't either. Agentic frameworks split work across specialised agents, the way a company splits it across roles.

Example client brief: an app that (1) answers questions over PDFs, (2) writes emails, (3) creates Jira tickets, (4) sends notifications via Teams, Slack or ServiceNow.

A company delivers that with roles: solution architect, lead developer, researcher, tester, project manager, customer support.

How a project flows (the human version agents imitate)

1. Client → intake team (requirements)
2. → project manager
3. → solution architect (decides which roles/resources are needed)
4. → researcher + developer build a UAT version
5. → tester validates it against a **golden dataset** (expected Q&A from the client)
6. → evaluation metrics (e.g. hallucination score) → final response delivered

Why a framework? It gives ready-made building blocks and a provider-agnostic interface. If the client switches OpenAI → Gemini → Claude, you swap the model, not rebuild the app (the same "one app for all your credit cards" analogy used for LangChain).

2. The four frameworks compared

Rule of thumb: LangChain for chatbots and RAG pipelines; LangGraph for complex, stateful, production workflows; CrewAI and AutoGen for role-based multi-agent collaboration.

Framework	Core idea	Best for	Example
LangChain	Ready-made components: loaders, splitters, embeddings, retrievers, chains, memory, output parsers	Simple LLM chatbots, RAG apps, PDF/SQL/CRM chatbots, email assistants	Resume screener: load 5,000 CVs → RAG → top 100 matches for a job description

Framework	Core idea	Best for	Example
LangGraph	Workflow as a **graph** of nodes + edges with shared **state**	Production-grade, complex workflows: routing, branching, loops, approvals	Hospital assistant that routes queries, detects sentiment and escalates
CrewAI	A "crew" of agents, each with role, goal, backstory, tools; tasks assigned to agents	Role-based teams; fast POCs; research → write → review pipelines	Travel planner; marketing (keyword research → SEO → writer → reviewer)
AutoGen / AG2	Conversable agents that chat with each other, with a user proxy for human input	Multi-agent conversations, coding agents, research automation; fits Microsoft stacks	Strategy agent ↔ content agent with human feedback (Section 3)

What makes LangGraph powerful

- **State** shared across nodes
- **Persistence/memory** via checkpointers (resume, history)
- **Retries** and **loops** ("not London? search again")
- **Conditional branching** (route by intent or sentiment)
- **Human-in-the-loop** approval steps
- **Checkpoints** to pause and resume
- **Parallel execution** of independent branches

Real-world LangGraph examples from class

1. **Hospital assistant:** understand query → route by intent (general / HR / IT / insurance / doctor) → detect sentiment (positive / neutral / negative) → if negative, an escalation matrix emails the customer an apology with a 24-hour turnaround (TAT) and escalates to the department. Critical patients trigger immediate doctor assistance.
2. **Loan approval:** read documents → check KYC (Aadhaar/PAN), credit (CIBIL) score, income, fraud history → classify low / medium / high risk → human approves → eligible amount.
3. **Insurance claims:** upload images → OCR → fraud detection → policy and cost-cap validation (e.g. room-rent limits) → manager approval → settlement.
4. **Smart search:** understand intent → route to PDF / SQL / MongoDB / internet agents → merge results → ask the human when needed → answer.

Why not one big prompt to one LLM?

Specialised agents see a smaller scope (just HR docs, say), so prompts stay focused and retrieval stays precise. Each agent is also easier to test and to swap out. The trade-off: more agents mean more LLM calls and orchestration overhead.

3. AutoGen POC: marketing strategy + content

Case study: a startup made a lightweight water bottle for school kids under 16, with a **$3,000** marketing budget. As marketing manager, find the best channels and create compelling content, using AutoGen agents.

Design: two task agents plus a **user proxy**. The user proxy acts like a project manager: it passes messages between agents and asks the human for feedback.

```
!pip install ag2[openai]      # "autogen" / AG2 package

import os, autogen
from getpass import getpass
os.environ["OPENAI_API_KEY"] = getpass("OpenAI API key: ")

llm_config = {"config_list": [{"model": "gpt-4o-mini", "api_key":
os.environ["OPENAI_API_KEY"]}]}

# Agent 1 – strategy
strategy_agent = autogen.AssistantAgent(
    name="strategy_agent",
    system_message=("You are a logical marketing manager tasked with crafting
a marketing strategy. "
                    "Your role is to gather details on the product, target
audience and budget, "
                    "then identify the best marketing channels to give the
product maximum visibility."),
    llm_config=llm_config,
)

# Agent 2 – content
content_agent = autogen.AssistantAgent(
    name="content_agent",
    system_message=("You are a content generator. Create engaging, channel-
specific content that aligns "
                    "with the marketing strategy, captures attention,
resonates with the target audience "
                    "and respects the allocated budget."),
    llm_config=llm_config,
)
```

```
# Coordinator / human-in-the-loop
user_proxy = autogen.UserProxyAgent(
    name="user_proxy",
    human_input_mode="ALWAYS",          # ask the human after every reply
    is_termination_msg=lambda x: x.get("content",
"").rstrip().endswith("TERMINATE"),
    code_execution_config=False,
)

product_details = ("Lightweight water bottle for school children under the
age of 16. "
                   "The marketing budget is USD 3,000.")
writing_task = "Write engaging content for the mentioned product and
channels."

results = autogen.initiate_chats([
    {"sender": user_proxy, "recipient": strategy_agent,
     "message": product_details, "summary_method": "last_msg"},
    {"sender": user_proxy, "recipient": content_agent,
     "message": writing_task, "summary_method": "last_msg"},   # strategy
summary is carried over
])
```

`human_input_mode` (interview question)

Mode	Behaviour	Use when
`ALWAYS`	Asks the human after every agent reply (press Enter to auto-reply, type feedback, or `exit` to end that chat)	Decisions need approval (payments, publishing)
`TERMINATE`	Asks the human only when a termination condition or the max auto-replies is reached	Mostly autonomous with a final check
`NEVER`	Never asks; agents run fully autonomously	Trusted, low-risk tasks

What happened in the demo

1. **Strategy agent's plan:** audience = children 6–16 (primary) and parents/educators (secondary). Budget: social ads $900 (Instagram/TikTok), influencers $900, content/blog $600, offline/school events $300, promotions $300.

2. **Human feedback:** "Spend $700 on social and $600 offline, and show a table" → revised table totalling $3,000.
3. `exit` → the user proxy passes the strategy to the **content agent**, which writes Instagram slides, influencer ideas, blog outline and offline promo copy.
4. **Feedback:** "more interesting, with emoji, more detail" → richer content → `exit`.

A third agent (e.g. one that writes an email to school principals) could be chained the same way.

4. CrewAI POC: research analyst + content writer

Task: research "AI in healthcare" from reliable web sources with citations, then turn the findings into an engaging blog post.

CrewAI in three steps: (1) create **Agents** (role, goal, backstory, tools, llm) → (2) assign **Tasks** (description, expected_output, agent) → (3) form a **Crew** (agents + tasks) and `kickoff()`.

```
!pip install crewai crewai-tools

import os
from getpass import getpass
from crewai import Agent, Task, Crew, LLM
from crewai_tools import SerperDevTool

os.environ["GEMINI_API_KEY"] = getpass("Gemini API key: ")
os.environ["SERPER_API_KEY"] = getpass("Serper API key: ")   # serper.dev,
2,500 free queries

llm = LLM(model="gemini/gemini-2.5-flash")   # provider prefix required
(LiteLLM)
search_tool = SerperDevTool()                 # live Google search results

# Step 1 – agents
research_analyst = Agent(
    role="Senior Research Analyst",
    goal="Research, analyze and synthesize comprehensive information on
{topic} from reliable web sources",
    backstory=("You are an expert research analyst with advanced web research
skills. You find, analyze "
               "and synthesize information using search tools, distinguish
reliable from unreliable "
               "sources, fact-check and cross-reference, and always provide
citations and sources."),
    tools=[search_tool],
```

```
    llm=llm,
    verbose=True,
)

content_writer = Agent(
    role="Content Writer",
    goal="Transform research findings into engaging blog posts while
maintaining accuracy",
    backstory=("You are an expert content writer who turns technical research
into engaging, actionable "
               "content. You keep every fact and citation, and make complex
topics approachable "
               "without oversimplifying."),
    llm=llm,
    verbose=True,
)

# Step 2 – tasks
research_task = Task(
    description=("1. Conduct comprehensive research on {topic}: recent
developments, key trends, "
                 "expert opinions, statistics.\n2. Evaluate source
credibility and fact-check.\n"
                 "3. Organize findings into a structured research brief.\n4.
Include all citations and sources."),
    expected_output=("A detailed research report with an executive summary,
trend analysis, verified "
                     "facts and statistics, citations with original links,
and clear sections/bullets."),
    agent=research_analyst,
)

writing_task = Task(
    description=("Using the research brief, create an engaging blog post that
turns technical "
                 "information into actionable content while keeping all facts
and citations."),
    expected_output="A polished Markdown blog post with clear sections and
emoji where helpful.",
    agent=content_writer,   # the class demo mistakenly reused
research_analyst here
)

# Step 3 – crew
crew = Crew(agents=[research_analyst, content_writer],
            tasks=[research_task, writing_task],
```

```
            verbose=True)

# Step 4 – run
result = crew.kickoff(inputs={"topic": "AI in healthcare"})
print(result)
```

Demo status: the run failed in class. Fixes for the errors seen:

- "Invoked sync logic within the running event loop" happens inside Jupyter/Colab. Use `await crew.kickoff_async(inputs=...)`, or run the code as a `.py` script.
- Gemini models need the `gemini/` prefix and `GEMINI_API_KEY`. Use **2.5 Flash** (not "flask").
- Assign `writing_task` to `content_writer` (bug spotted by a student).
- Fast-moving libraries (crewai, litellm) can break on day-one releases. Pin a known-good version if needed.

5. Vibe-coding tools shown in class

These tools generate output from a prompt, but you still need the concepts to explain, debug and defend the result to a client.

Tool	What it does	Demo
Napkin AI	Turns text into presentation visuals	Made financial-overview slides from a one-line prompt
Windsurf	AI code editor/agent (like Cursor or GitHub Copilot)	Built a full Titanic ML pipeline (preprocessing → EDA → split → decision tree & random forest → evaluation → Streamlit app) from one prompt
GitHub Copilot, Cursor, Replit, Gamma	Code and presentation generation	Covered later

Lecturer's point: companies hire people who can debug and justify generated code, e.g. explain why age is imputed with the median and "Embarked" with the mode. Install Windsurf (free tier) and try it.

6. Q&A highlights

Question	Answer
LangChain can also search the internet. Why LangGraph?	LangChain chains are linear and get unwieldy for routing, loops and approvals. LangGraph models these explicitly with state and conditional edges, which makes it more reliable and maintainable for complex flows.
Router system vs. multi-agent system?	A router classifies the request (intent, sentiment) and sends it down one branch. A multi-agent system is the set of specialist agents those branches lead to. Routers are often one node inside a multi-agent graph.
How do we feed Salesforce/enterprise data to agents?	Build agentic RAG: connectors/tools per source (RDBMS, JSON, PDFs), agents that know which source to use ("agent skills"), and retrieval as a tool.
How do we track agent interactions?	Tracing/observability (e.g. LangSmith), covered later.
CrewAI or AutoGen?	Either for role-based multi-agent work. AutoGen fits Microsoft/Azure stacks; CrewAI is quick for POCs. Production-heavy flows often use LangGraph.
Is the LangGraph planner/router an LLM?	Usually yes: an LLM classifies the intent (often with structured output), and conditional edges route on that result. Simple cases can use rules.
Why not one LLM with a big system prompt?	Possible for small apps. Specialised agents keep context focused, are easier to test, and scale across departments.
Where does memory live in LangGraph?	In the graph state, persisted by a checkpointer (e.g. `MemorySaver`), plus a store for long-term memory.
Are VS Code's agent features the same thing?	Same concepts, packaged as low-code/vibe-coding tools. Covered later.

Question	Answer
What does `human_input_mode` do?	See Section 3: ALWAYS / TERMINATE / NEVER.
Can a third agent be added?	Yes. Add another chat to `initiate_chats` (AutoGen) or another Agent + Task (CrewAI).

7. Expert corrections to the lecture

1. **The feature list belongs to LangGraph.** State, memory, retries, loops, conditional branching, human approval, checkpoints and parallel execution were repeatedly read out as "LangChain" features. They describe LangGraph.
2. **LangGraph memory isn't automatic.** It comes from graph state plus a **checkpointer** you configure (e.g. `MemorySaver`, a Postgres saver) and optional long-term stores.
3. **Routers usually use an LLM.** Saying the router is "not an LLM, it's tool calling" is misleading. Typically an LLM classifies the intent (often via tool calling or structured output) and LangGraph's conditional edges route on it. Rule-based routing is the simpler alternative.
4. **`human_input_mode` semantics:** NEVER = fully autonomous (you still see output if verbose). TERMINATE = human asked only at termination or the max auto-reply limit. ALWAYS = human asked every turn. The lecture's descriptions of TERMINATE and NEVER were muddled.
5. **AutoGen vs. AG2:** AutoGen is Microsoft's framework. AG2 is a community fork of AutoGen 0.2 by its original creators (installed via `pip install ag2` / `autogen`). Microsoft's current AutoGen (0.4+) has a different API.
6. **`initiate_chats` takes a list of chat dicts** (sender, recipient, message, summary_method). The strategy result carries over to the next chat.
7. **Serper is a third-party Google Search API service,** not a Google product, and it isn't MCP. It's a search tool the agent calls.
8. **CrewAI with Gemini needs the `gemini/` model prefix** and `GEMINI_API_KEY`; the Colab "sync in running loop" error is solved with `kickoff_async`.
9. **"CrewAI/AutoGen are only for POCs"** is the lecturer's opinion. Both are used in production too; choose by team fit and control needs.
10. **Multi-agent isn't automatically cheaper or faster.** Narrower context helps, but every extra agent adds LLM calls and latency.

11. **The LangChain flow description was garbled.** The *application* orchestrates: query → retriever (embeddings + vector DB) → prompt → LLM. The LLM doesn't "connect to" the vector DB, and "API → Google search → embedding" isn't part of RAG.
12. **LangChain launched in late 2022** (not 2023).
13. **Windsurf** was acquired by Cognition in 2025 (Google hired its leadership and licensed its technology). It wasn't simply "acquired by Google".

8. Interview quick revision and homework

30-second answers

> **Which agent framework would you use?** LangChain for straightforward chatbots and RAG pipelines. LangGraph when the workflow needs routing, loops, retries, human approval and persistent state, which covers most production enterprise flows. CrewAI or AutoGen when the problem maps naturally to a team of role-based agents collaborating, and for quick POCs.

> **How do AutoGen agents collaborate?** Assistant agents each have a system message and LLM config. A UserProxyAgent sends tasks, relays results between agents, and brings in a human depending on `human_input_mode` (ALWAYS / TERMINATE / NEVER). `initiate_chats` runs sequential chats and carries summaries forward.

> **What are CrewAI's building blocks?** Agent (role, goal, backstory, tools, llm) → Task (description, expected_output, agent) → Crew (agents, tasks) → `kickoff(inputs=...)`.

Homework

- [ ] Re-run the AutoGen notebook; add a third agent (e.g. an email writer) and try each `human_input_mode`
- [ ] Fix and run the CrewAI notebook (correct agent assignment, Gemini prefix, `kickoff_async`)
- [ ] Sign up for Serper (2,500 free searches) and Windsurf
- [ ] Complete the weekly agent case studies the lecturer will share

Coming next

LangGraph hands-on (state, nodes, conditional edges, memory), then agentic RAG, Graph RAG, CRAG, fine-tuning (PEFT, LoRA/QLoRA), evaluation, observability, guardrails, responsible AI and MCP.

SESSION 8

Sunday, 28 June 2026

Project: Video Compliance Auditor on Azure

Project Notes – Session 8 (28 June 2026): Video Compliance Auditor on Azure

Oct 14, 2026 · @VIjay

1. Project brief

Build an agentic AI system that watches a video, checks it against compliance rules, and returns a structured JSON violation report, with no human watching. This is the first of 5–10 real-time, production-grade projects in this batch. It assumes you know prompt engineering, RAG, agentic RAG and LangGraph.

The problem

- Auditors manually watch hours of footage (YouTube, CCTV, call recordings, ads) looking for regulatory violations, fraud, policy breaches or operational mistakes.
- At scale (e.g. 5,000 hours) this is **slow** and **expensive**: many people and much time.

The solution

1. **Ingest** a video (e.g. a YouTube URL).
2. **Extract** what was said (transcript) and what's shown on screen (OCR text) with Azure Video Indexer.
3. **Check** that content against a compliance-rules knowledge base using RAG and LLM reasoning, orchestrated by LangGraph.
4. **Report** in JSON: which violation, where (timestamp), and why.
5. **Observe** every step: LangSmith traces the reasoning; Azure Application Insights monitors system health.

Where this applies

Banking and insurance (fraud and call monitoring), healthcare (procedure compliance), media (content moderation, misleading claims), audit and consulting firms (Big Four). The same pattern works for PDFs and reports: "summarise these and flag rule breaches".

One-line pitch: an agentic AI system that automatically analyses videos, checks compliance rules, and generates a violation report without human effort.

2. Tech stack

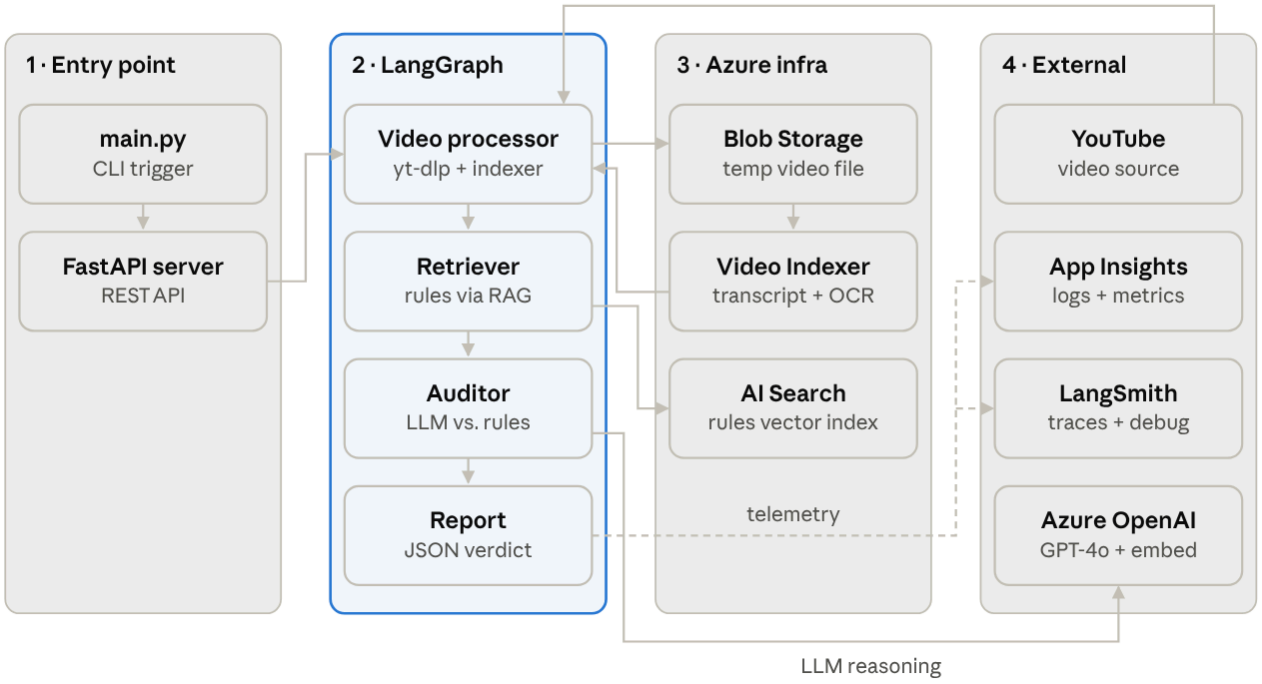
Layer	Service / tool	Job in this project
Storage	**Azure Blob Storage**	Holds the raw/temporary video file (and rule PDFs)
Video understanding	**Azure Video Indexer**	Speech-to-text transcript, on-screen text (OCR), timestamps, speakers, scenes/keyframes
LLM	**Azure OpenAI** (via Azure AI Foundry): GPT-4o	Reasoning: compares content against rules, writes the verdict
Embeddings	Azure OpenAI `text-embedding-3-small`	Turns rule documents and queries into vectors
Knowledge base	**Azure AI Search**	Vector index of compliance rules (the RAG store)
Orchestration	**LangGraph** (+ LangChain components)	Stateful workflow: nodes, edges, conditional routing
LLM observability	**LangSmith**	Traces every reasoning step; logs and debugging (free tier: 5,000 traces/month)
System monitoring	**Azure Application Insights** (via OpenTelemetry)	Health checks, request logs, metrics, failures
API layer	**FastAPI**	Production web API that triggers the audit (Streamlit is for POCs)
Language / tooling	Python, `uv`, VS Code	Project and dependency management

Use GPT-4o or any newer deployed model.

3. Architecture

The system has four phases: an entry point triggers a LangGraph workflow, which uses Azure services for storage, video understanding and search, plus external services for the LLM and observability.

Video in, JSON compliance verdict out



Video compliance auditor · 4 phases

Read it left to right. The API starts the graph. The processor pulls the video into Blob Storage and gets transcript + OCR back from Video Indexer. The retriever fetches the relevant rules. The auditor reasons with GPT-4o. The report is emitted as JSON, with telemetry to App Insights and LangSmith (dashed lines).

Phase	Components	Role
1. Entry point	`main.py` (CLI trigger), FastAPI server	Starts an audit for a video URL
2. Orchestration (LangGraph)	Video processor → Retriever → Compliance auditor → Report	Stateful workflow: nodes, edges, conditional routing
3. Azure infrastructure	Blob Storage, Video Indexer, AI Search	Temporary video storage, transcript + OCR extraction, rules vector index
4. External + observability	YouTube source, Azure OpenAI, App Insights, LangSmith	Input, LLM + embeddings, system metrics, reasoning traces

Pipeline steps

1. **Video input:** CCTV, call recordings or a YouTube URL; the raw source data.

2. **Extraction:** Video Indexer converts unstructured video into structured text (transcript + OCR + timestamps). LLMs can't reliably reason over raw video at scale, so this step matters, much like load → chunk → embed in RAG.
3. **Preprocessing:** clean and merge transcript and OCR text.
4. **LangGraph workflow:** controls the step-by-step logic and AI decision flow (state, nodes, edges, conditional edges, start/end).
5. **Rule check:** retrieve the relevant rules from AI Search; GPT-4o compares content against them.
6. **Decision:** compliant or not; identify each violation.
7. **Report:** structured JSON compliance report.
8. **Observe:** LangSmith traces the reasoning; App Insights monitors health.

4. Project setup

Install uv and create the project

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
# Windows PowerShell (the curl command fails there)
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 |
iex"

cd Desktop
mkdir genai-pipeline
cd genai-pipeline
uv init --app        # creates .gitignore, .python-version, main.py,
pyproject.toml, README.md
```

Folder structure

```
mkdir backend
cd backend
mkdir data, scripts, src, tests        # PowerShell: comma-separated; bash:
mkdir data scripts src tests
mkdir src/api, src/graph, src/services
```

```
genai-pipeline/
├── main.py                 # CLI trigger (entry point)
├── pyproject.toml
├── .env                    # secrets – never commit
└── backend/
    ├── data/               # rule PDFs / sample inputs
    ├── scripts/
    │   └── index_documents.py   # builds the Azure AI Search index
    ├── src/
    │   ├── __init__.py
    │   ├── api/            # FastAPI server
    │   ├── graph/          # LangGraph state, nodes, workflow
    │   └── services/       # Video Indexer, storage, search clients
    └── tests/
```

`.env` template

```
# Storage
AZURE_STORAGE_ACCOUNT_NAME=""

# Azure OpenAI (LLM + embeddings)
AZURE_OPENAI_API_KEY=""
AZURE_OPENAI_ENDPOINT=""
AZURE_OPENAI_API_VERSION=""
AZURE_OPENAI_CHAT_DEPLOYMENT="gpt-4o"
AZURE_OPENAI_EMBEDDING_DEPLOYMENT="text-embedding-3-small"

# Azure AI Search (vector knowledge base)
AZURE_SEARCH_ENDPOINT=""
AZURE_SEARCH_API_KEY=""
AZURE_SEARCH_INDEX_NAME=""

# Azure Video Indexer
AZURE_VI_NAME=""
AZURE_VI_LOCATION=""
AZURE_SUBSCRIPTION_ID=""
AZURE_VI_ACCOUNT_ID=""
AZURE_RESOURCE_GROUP=""

# Monitoring
APPLICATIONINSIGHTS_CONNECTION_STRING=""

# LangSmith tracing
```

```
LANGCHAIN_TRACING_V2="true"
LANGCHAIN_API_KEY=""
LANGCHAIN_ENDPOINT="https://api.smith.langchain.com"
LANGCHAIN_PROJECT="video-compliance-auditor"
```

Where to find each value: the resource's **Keys and Endpoint** page (OpenAI, AI Search), the Video Indexer resource overview (name, location, account ID, subscription, resource group), and the Application Insights overview (connection string).

5. Q&A highlights

Question	Answer
Does Video Indexer read the video straight from Blob Storage?	Yes. The video sits temporarily in Blob Storage, and Video Indexer processes it from there and returns insights.
Is video processed frame by frame? A 3-hour video is a lot of frames.	Video Indexer samples keyframes and scenes rather than every frame. It runs as a managed Azure service, so you don't need your own GPU. Cost and processing time grow with video length.
Blob Storage vs. AI Search index?	Blob Storage holds raw files. AI Search holds the processed, embedded, searchable index. They are separate services.
If the video is already in Blob Storage, why connect the orchestrator to the YouTube source?	The video-processor node downloads from the source and uploads to Blob Storage. After that, processing runs from Blob Storage and Video Indexer insights.
What is the "compliance auditor"?	The node that applies the rule set (guardrails) to the extracted content and produces the verdict.
Where does testing and evaluation happen?	Unit and integration tests in `tests/`. Runtime metrics in App Insights, reasoning traces in LangSmith. Evaluation is covered later.
How do I remember all this for interviews?	Feynman technique: explain the architecture in simple (even local-language) terms, find the gaps, re-study, repeat. Redraw the diagram from memory.

Question	Answer
Schedule	Project track: Saturday and Sunday, 7–9 PM, continuing this project.

6. Expert corrections to the lecture

1. **FastAPI is a backend API framework,** not the front end, and OpenTelemetry is not a front end either. It's the telemetry standard that ships traces and metrics to Application Insights. Any UI (React, Streamlit) would call the FastAPI endpoints.
2. **Transcript and OCR are different signals.** Transcript = speech-to-text of what's *said*. OCR = text *visible on screen* (captions, banners, product claims). Video Indexer also returns labels, scenes, brands and timestamps.
3. **"Audit 5,000 hours in a second" overstates it.** Video indexing takes real processing time per video (minutes per hour of footage). The win is parallel, automated review at a fraction of human cost and time.
4. **Skippable YouTube ads aren't compliance violations.** Real examples: misleading health or financial claims, missing disclosures ("sponsored", risk warnings), prohibited content, or claims that breach brand or regulatory guidelines.
5. **The guardrails page shown was Amazon Bedrock Guardrails.** On Azure, the equivalent is **Azure AI Content Safety** (plus your own rule base in AI Search).
6. **Use `yt-dlp` (not "YouTube DLP").** Downloading YouTube videos can breach YouTube's terms. In real projects, use videos you own or have rights to, or files the client provides.
7. **Shell commands:** the `curl … | sh` uv installer is for macOS/Linux; use the PowerShell installer on Windows. PowerShell's `mkdir` needs comma-separated names, which caused the errors in class. The command is `uv init --app`.
8. **The LangSmith env var is `LANGCHAIN_TRACING_V2=true`** (or the newer `LANGSMITH_TRACING`, `LANGSMITH_API_KEY`).
9. **Never commit `.env`.** Keep it in `.gitignore`; in production, store secrets in **Azure Key Vault** or use managed identities.
10. **"Azure Analysis Services" isn't part of this stack.** The monitoring service is Application Insights (part of Azure Monitor).

7. Interview pitch and prep for next session

60-second project pitch

> I built an agentic video compliance auditor on Azure. A FastAPI endpoint, or a CLI trigger, starts a LangGraph workflow. The first node downloads the video to Blob Storage and runs Azure Video Indexer to extract the transcript and on-screen OCR text with timestamps. A retriever node queries an Azure AI Search vector index of compliance rules, embedded with text-embedding-3-small. An auditor node uses GPT-4o to compare the content against those rules and outputs a structured JSON report: each violation, its timestamp, and the reason. LangSmith traces every reasoning step, and Application Insights (via OpenTelemetry) monitors health and latency. It replaces hours of manual review with an automated, auditable pipeline.

Before next session

- [ ] Create a **LangSmith** account (free Developer plan) and an API key
- [ ] Create or activate an **Azure** subscription (free trial or pay-as-you-go)
- [ ] Provision: Storage account, Azure OpenAI (deploy GPT-4o + text-embedding-3-small), AI Search, Video Indexer, Application Insights
- [ ] Fill in your `.env` and set up the folder structure with `uv`
- [ ] Redraw the architecture diagram from memory (e.g. in draw.io)

Next session

Wire up the Azure services, build the LangGraph state, nodes and edges, index the rule documents, run the pipeline, and inspect LangSmith traces and App Insights metrics.